Frontiers of Intelligence

Filippo Lentoni

September 29, 2026

Contents

1 The Anatomy of an Agent **2**
- 1.1 The Four Layers of an Agent . . . 2
- 1.2 From Anatomy to Implementation . . . 3

I The Intelligence Layer **4**

2 Foundation Models **5**
- 2.1 Tokenisation: the First Compression Step . . . 5
- 2.2 Embeddings and Representation Geometry . . . 5
- 2.3 The Limitations of Sequential Models . . . 5
- 2.4 Self-Attention . . . 6
- 2.5 The Original Transformer . . . 6
- 2.6 From Encoder-Decoder to Decoder-Only Models . . . 6
- 2.7 Positional Information . . . 7
- 2.8 Inference: Logits, Decoding, and Structured Output . . . 7
- 2.9 Why This Matters for Agents . . . 7
- 2.10 Summary . . . 7

3 Frontier Model Capabilities **8**
- 3.1 Open-Weight and Closed Models . . . 8
- 3.2 The Original Transformer . . . 8
- 3.3 The Modern Frontier Decoder . . . 9
- 3.4 How the Field Evolved . . . 9
- 3.5 Summary . . . 10

4 Cognitive Architecture **11**
- 4.1 The Agent Loop . . . 11
- 4.2 Clarification and Autonomy . . . 11
- 4.3 Stopping Conditions . . . 12
- 4.4 Summary . . . 12

II The Action Layer **13**

5 The Action Layer **14**
- 5.1 Schemas as Behavioural Contracts . . . 14
- 5.2 Read Tools and Write Tools . . . 15

5.3 Agent-as-Tool and the Opacity Problem . . . 15
5.4 Summary . . . 16

6 Tools, Function Calling, and MCP 17
6.1 The Function-Calling Loop . . . 17
6.2 MCP Primitives . . . 17
6.3 Local Tools and Remote Tools . . . 18
6.4 Validation Before Execution . . . 18
6.5 Tool Hallucination . . . 18
6.6 Summary . . . 18

III The Memory and Knowledge Layer 19

7 The Memory and Knowledge Layer 20
7.1 Why Memory and Knowledge Form a Single Layer . . . 20
7.2 Types of Memory . . . 20
7.3 Skills as Procedural Knowledge . . . 21
7.4 Context Compression . . . 21
7.5 Memory Write Policy . . . 22
7.6 Retrieval-Augmented Generation . . . 22
7.7 Summary . . . 22

IV The Security and Governance Layer 23

8 The Security and Governance Layer 24
8.1 The Attack Surface . . . 24
8.2 Identity, Authentication, and Authorisation . . . 24
8.3 Prompt Injection . . . 25
8.4 Guardrails . . . 25
8.5 Data Flow Control . . . 26
8.6 Human Approval . . . 26
8.7 Sandboxing . . . 26
8.8 Governance and Auditability . . . 26
8.9 Summary . . . 27

V Applications: Building Agents in Production 28

9 Building Agents: Strands and LangGraph 29
9.1 Step 1: Define the tools . . . 29
9.2 Step 2: Load gateway tools over MCP . . . 30
9.3 Step 3: Encode operating procedures in SKILL.md . . . 32
9.4 Step 4: Build the Strands agent . . . 33
9.5 Step 5: Build the LangGraph agent . . . 35
9.6 Step 6: The entrypoint and session identity . . . 38
9.7 Step 7: The full picture . . . 40
9.8 Framework comparison . . . 40

9.9 When to choose each framework . . . 41
9.10 Summary . . . 41

10 From local to the cloud: Deploying Agents with AWS AgentCore 42
10.1 Step 1: Understand what the runtime expects . . . 42
10.2 Step 2: Build the tool layer . . . 43
10.3 Step 3: Create the execution role . . . 43
10.4 Step 4: Provision AgentCore Memory . . . 44
10.5 Step 5: Create the AgentCore Gateway . . . 45
10.6 Step 6: Package and deploy the Runtime . . . 46
10.7 Step 7: Add the proxy Lambda for browser clients . . . 47
10.8 Step 8: Deploy and verify . . . 48
10.9 What the runtime owns versus what it delegates . . . 49
10.10Strands and LangGraph in the same runtime . . . 49
10.11Summary . . . 50

11 Evaluation 51
11.1 Traces . . . 51
11.2 Scores . . . 51
11.3 Datasets and Experiments . . . 52
11.4 Stratified Sampling with Unbalanced Weights . . . 52
11.5 LLM-as-a-Judge . . . 53
11.6 Failure Taxonomy . . . 54
11.7 Offline and Online Evaluation . . . 54
11.8 Summary . . . 54

VI Building an AI-First Company 55

12 Building an AI-First Organization 56
12.1 Minimum Architecture . . . 57
12.2 Organizational Structure . . . 57
12.3 Competitive Advantage . . . 58

VII Mathematical Foundations 59

13 From Hilbert Spaces to σ-Algebras: The Transformer Explained at a Higher Level of Mathematical Abstraction 60
13.1 Introduction . . . 60
13.2 Mathematical Tools . . . 61
13.2.1 Vector spaces, inner products, and Hilbert spaces . . . 62
13.2.2 Operators: linear, nonlinear, and piecewise . . . 62
13.2.3 Spectral theory in finite dimensions . . . 63
13.2.4 Banach spaces, compact operators, and the spectral theorem . . . 64
13.2.5 Weak derivatives and Sobolev spaces . . . 66
13.2.6 Measure, probability, and the σ-algebra of events . . . 67
13.2.7 The softmax and Gibbs measures . . . 68
13.2.8 Fourier analysis and positional structure . . . 69

13.2.9 Finite-precision arithmetic . . . 70
13.2.10 Information theory . . . 70
13.2.11 Approximation theory . . . 71
13.3 The Original Transformer . . . 71
13.3.1 The task and the overall architecture . . . 72
13.3.2 Scaled dot-product attention . . . 72
13.3.3 Multi-head attention . . . 72
13.3.4 Three uses of attention . . . 73
13.3.5 Feed-forward networks, embeddings, and the output layer . . . 73
13.3.6 Positional encoding . . . 73
13.3.7 Why self-attention . . . 73
13.3.8 Training and decoding . . . 74
13.3.9 Results . . . 74
13.4 The Transformer as Operators and Measures . . . 74
13.4.1 The Transformer as a composition of operators . . . 74
13.4.2 The operators one by one . . . 76
13.4.3 Attention as a data-dependent Markov kernel . . . 77
13.4.4 Spectral structure: heads, weight tying and positional rotations . . . 79
13.4.5 The last layer: a probability measure on the σ-algebra of token events . . . 80
13.4.6 Sequences: from one token to a measure on translations . . . 82
13.4.7 Decoding and the idealized model . . . 83
13.5 Beyond the Structure of the Family . . . 84
13.6 Sources . . . 84

Conclusion **86**

About the Author

Filippo is a Senior Applied Scientist at Amazon, specialized in agentic planning and large-scale, compliance-critical machine learning systems. He is a visiting lecturer at Columbia Engineering, where he teaches theoretical introduction to AI Agents, and has collaborated with the Technical University of Munich as Science Lead on post-training research since 2023. Filippo holds a Master's degree in Mathematical Engineering and Statistical Learning from Politecnico di Milano. His work sits at the intersection of statistical learning theory, agentic AI, and the design of large-scale AI systems.

The idea for this book emerged from a conversation with Sherwin Wu, OpenAI's Head of Engineering, at UC Berkeley in 2025. Sherwin argued that despite the investment behind Operator, Deep Research, the Responses API, o3, and Codex, industry adoption of agentic AI remained limited. The core insight was that the primary blockers were not only intelligence-related, but deployment-related: models were becoming increasingly capable, yet the engineering infrastructure, evaluation discipline, and documentation required to deploy agents reliably were still immature. This book is an attempt to address that gap.

Chapter 1

The Anatomy of an Agent

This book begins with a simple question: what is an AI agent? The anatomy of an agentic AI system comprises four components: the intelligence layer, the action layer, the memory and knowledge layer, and the security layer. These layers are conceptually distinct but continuously interdependent in production: the model reasons over context assembled by the memory layer, selects actions exposed by the action layer, and operates within boundaries enforced by the security layer. Evaluation and observability sit across all four, measuring whether the full system behaves correctly at each step and over time.

1.1 The Four Layers of an Agent

The first layer is the *intelligence layer*. In this book, the intelligence layer encompasses two tightly coupled components: the foundation model, which provides the core reasoning and generation capability, and the cognitive architecture, which governs how the model pursues a goal across multiple steps, deciding when to act, when to retrieve, when to seek clarification, and when to stop. The subsequent chapter explains how these models work, beginning with the original Transformer architecture and progressing toward the design choices that characterise modern frontier models.

The second layer is the *action layer*. This layer enables the agent to produce effects beyond generating text. It encompasses function calling, tools, MCP servers and agent-as-tool patterns. Some tools are deterministic, such as a calculator, while others are stochastic, such as a sub-agent that performs research, writes code, or produces a judgment. Together they define the set of effects the agent can produce in the world beyond generating text.

The third layer is the *memory and knowledge layer*. This layer determines what information is available to the model at the moment of decision. It includes short-term conversation state, long-term user memory, retrieval-augmented generation, knowledge bases and reusable skills. In this book, skills are treated as a form of procedural knowledge: versioned, retrievable packages of instructions or patterns that improve the reliability with which the agent performs recurring tasks.

The fourth layer is the *security layer*. An agent with access to tools, persistent memory, and the ability to take actions requires security as a foundational architectural concern. The security layer includes identity, authentication, authorisation, permission boundaries, guardrails, human approval mechanisms, audit logging, sandboxing, and protection against prompt injection and data exfiltration. The scope and rigour of this layer should scale with the autonomy and consequence of the agent's permitted actions.

1.2 From Anatomy to Implementation

The architecture described above provides the conceptual foundation for the rest of the book. The chapters that follow develop each layer in depth: how foundation models are trained and how they reason, how the action layer is designed and governed, how the memory and knowledge layer is constructed and evaluated, and how the security layer is enforced in production. The application section then demonstrates how these layers compose into an end-to-end system: an agent built with LangGraph, connected to tools and memory, evaluated with Langfuse, and iterated through a pipeline that closes the loop between offline evaluation, production traces, human feedback, and future evaluation datasets. The goal is to give the reader the conceptual and practical tools to build an agent that can be tested, monitored, improved, and deployed through a governed production pipeline in their own domain.

Part I

The Intelligence Layer

Chapter 2

Foundation Models

The Transformer architecture is the substrate on which modern language models and therefore agentic AI systems are built. This chapter explains the modelling foundations needed for the rest of the book: how text becomes tokens, how tokens become vectors, and why decoder-only Transformers became the default architecture for general-purpose assistants.

2.1 Tokenisation: the First Compression Step

Before a model can process text, that text must be converted into tokens: discrete units such as words, subwords, bytes, or characters. Tokenisation shapes cost, context length, and multilingual behaviour.

Subword tokenisation, implemented through algorithms such as Byte-Pair Encoding, is the approach used by most modern language models. Common words are represented as a single token; rare words are decomposed into reusable subword pieces; and out-of-vocabulary terms are handled gracefully by decomposing them into familiar subword units, balancing vocabulary size against sequence length more effectively than word-level or character-level alternatives.

For agent builders, tokenisation has direct engineering consequences: the cost and context capacity of any operation scale with token count.

2.2 Embeddings and Representation Geometry

After tokenisation, each token is mapped to a dense vector called an embedding. Dense embeddings place tokens in a learned vector space where similar meanings tend to occupy related regions, expressing semantic relatedness as geometric proximity.

The same principle underlies retrieval. A document chunk and a query can both be embedded into the same vector space, and similarity in that space selects relevant context. Both language models and retrieval-augmented generation systems rest on the assumption that meaning can be approximated by geometry in a learned representation space.

2.3 The Limitations of Sequential Models

Before Transformers, recurrent models such as RNNs and LSTMs processed text sequentially, updating a hidden state token by token. This introduced order sensitivity but imposed two significant constraints: long-range dependencies were difficult to maintain as information from early

tokens could fade across the sequence, and training was difficult to parallelise because later hidden states depended on earlier ones.

2.4 Self-Attention

Self-attention allows each token to directly compare itself to every other token in the sequence. For each position, the model constructs three learned projections: a query, a key, and a value, where the query represents what the token is looking for, the key represents what another token offers for matching, and the value represents the content to be aggregated.

Scaled dot-product attention is defined as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V.$$

The matrix $QK^\top$ contains pairwise relevance scores between query and key positions; the softmax converts those scores into a probability distribution; multiplying by V produces a weighted combination of value vectors; and the division by $\sqrt{d_k}$ stabilises the scale of dot products as dimensionality grows.

Multi-head attention repeats this operation with multiple learned projections in parallel, allowing the model to attend simultaneously to local syntactic structure, entity references, and long-range answer spans, producing representations that are context-aware across multiple dimensions at once.

2.5 The Original Transformer

The original Transformer, introduced in *Attention Is All You Need*, used an encoder-decoder architecture. The encoder processed the source sequence with full bidirectional self-attention; the decoder generated the target sequence autoregressively through masked self-attention over prior outputs and cross-attention over encoder representations. Chapter 13 restates this architecture as a composition of operators and a Markov kernel, for readers who want the full mathematical treatment.

The architecture established the design patterns that remain central to modern models: attention sublayers for token-to-token interaction, position-wise feedforward sublayers for nonlinear transformation, residual connections for gradient flow, normalisation for training stability, positional encodings to supply order information that permutation-invariant attention cannot recover, and causal masking to enforce left-to-right generation. Together these choices gave every token a direct path to every other token and expressed the computation as matrix operations that GPUs can parallelise efficiently.

2.6 From Encoder-Decoder to Decoder-Only Models

Three Transformer families emerged from the original design. Encoder-only models such as BERT apply bidirectional self-attention to produce strong contextual representations for classification, extraction, and ranking. Encoder-decoder models such as T5 retain the full sequence-to-sequence structure for tasks requiring one sequence to be transformed into another. Decoder-only models became dominant for large generative models because next-token prediction is simple, scalable, and supports open-ended generation.

Decoder-only models are the architecture of choice for agent systems because a conversation, a plan, a tool call, a tool result, and a final response can all be serialised into a single sequence that the model learns to continue in the desired format.

2.7 Positional Information

Self-attention is permutation-invariant: the same scores result regardless of token order unless positional information is explicitly introduced. Early Transformers added absolute positional encodings to token embeddings; modern frontier models use Rotary Positional Embeddings (RoPE), which encode position through the attention computation itself and generalise more effectively to long sequences.

2.8 Inference: Logits, Decoding, and Structured Output

At inference time, a decoder-only model produces a distribution over the vocabulary at each step as a vector of logits. Greedy decoding selects the highest-probability token; sampling introduces stochasticity controlled by temperature, top-k, and top-p parameters.

2.9 Why This Matters for Agents

Every agentic capability is built on these modelling primitives: tool calls are generated token sequences, retrieved documents and memory summaries are context tokens, plans are generated text, and cost and latency scale with total token count. Understanding Transformers at this level supports the engineering decisions that follow in every subsequent chapter: how to manage context, when to constrain generation, why retrieval precision matters, and how model capability translates into agent reliability.

2.10 Summary

Transformers replaced sequential recurrence with parallel self-attention, giving every token a direct path to every other token in a form that modern hardware can execute efficiently. Decoder-only Transformers became the default for large generative models because next-token prediction scales cleanly and supports open-ended generation. Agents are built by placing this generative engine inside a stateful loop with tools, memory, context engineering, and security boundaries, the layers this book addresses in the chapters that follow.

Chapter 3

Frontier Model Capabilities

A frontier model is a model near the leading edge of capability at a given moment. The term refers to proximity to the current frontier across one or more dimensions: reasoning, coding, tool use, multimodality, long-context understanding, instruction following, safety behaviour, latency.

For agent builders, frontier models matter because they determine which workflows can be automated with acceptable reliability.

For example, for agentic coding, SWE-bench evaluates whether a frontier model can resolve real GitHub issues by modifying code and passing the repository's own test suite.

3.1 Open-Weight and Closed Models

The model ecosystem is often described as a competition between open and closed models, but the terminology requires precision. Closed or proprietary models are those whose weights are withheld from public release. Users access them through an API or hosted product, and the provider controls serving, safety behaviour, updates, and pricing. Open-weight models are those whose weights are released, typically under a licence. Users can inspect, host, quantise, and in many cases fine-tune them.

Closed models are often the most direct path to frontier capability. Open-weight models offer control, data privacy, cost optimisation, and customisation. In practice, many deployments use both: a closed model for reasoning-intensive paths and an open-weight model for high-volume internal workflows where cost and latency are primary constraints.

3.2 The Original Transformer

The architectural foundation worth understanding is the original Transformer, introduced by Vaswani et al. in 2017. It comprised an encoder and a decoder, both built from multi-head self-attention, position-wise feedforward networks, residual connections, layer normalisation, and positional encodings. The encoder processed the full input sequence with bidirectional self-attention. The decoder generated output autoregressively, using masked self-attention over its own prior outputs and cross-attention over the encoder's representations.

The original design targeted sequence-to-sequence tasks such as machine translation rather than general-purpose language modelling. Its lasting contribution was the attention mechanism as a content-dependent communication primitive: queries and keys determine where the model attends; values carry the content aggregated at each position. This separation of relevance from content is

what allows a later token to use an earlier definition, a function call to attend to its declaration site, or a question to locate its answer span across a long document.

3.3 The Modern Frontier Decoder

Contemporary frontier models are almost universally decoder-only, but the internal block has been substantially refined from the original design. The field has converged on a representative set of architectural components, reflected across both closed models such as GPT, Claude, and Gemini, and open-weight families such as Llama, Qwen, Mistral, and DeepSeek.

Causal self-attention replaces the encoder-decoder cross-attention structure, with the model attending only to prior positions during generation. Rotary Positional Embeddings (RoPE) encode position through rotation applied to query and key vectors rather than through additive absolute embeddings, generalising more effectively to sequence lengths beyond those seen during training. RMSNorm, applied before each sub-layer rather than after, provides training stability with lower computational overhead than the original LayerNorm. Grouped-Query Attention, which has become the default for most large models, shares key and value projections across groups of query heads, substantially reducing the memory footprint of the KV cache during inference without significant loss in model quality. SwiGLU, combining a Swish activation with a gated linear unit, has replaced earlier activation functions in the feedforward sub-layer and is now the de facto standard. FlashAttention-style kernels reorganise the attention computation to minimise memory movement between GPU SRAM and HBM, reducing both memory usage and wall-clock time. Sparse Mixture-of-Experts feedforward layers activate only a subset of specialised sub-networks for each token, allowing a large total parameter count with a much smaller active parameter count per forward pass. Multimodal encoders or adapters extend the decoder to accept image, audio, or video inputs alongside text.

3.4 How the Field Evolved

The trajectory from the original Transformer to current agentic deployments can be read as a sequence of distinct eras, each expanding the scope of what language models could do.

The representation era was defined by encoder-only models such as BERT, which produced strong contextual embeddings through bidirectional attention and achieved state-of-the-art results on classification, extraction, and ranking tasks. These models established the value of deep contextual representations but were designed for understanding rather than generation.

The text-to-text era unified diverse tasks under a single sequence-to-sequence interface. Models such as T5 reframed translation, summarisation, question answering, and classification as instances of the same text-in, text-out paradigm, demonstrating that a single model architecture and training objective could cover a broad range of downstream applications.

The autoregressive scaling era established that next-token prediction on massive corpora, combined with scaling of data, parameters, and compute, could produce broad and transferable capabilities. The GPT family demonstrated this trajectory, and the empirical scaling laws that emerged from this period shaped research investment across the field.

The instruction-following era introduced supervised fine-tuning and preference optimisation, including reinforcement learning from human feedback, to align base models with human instructions. The model's task shifted from text continuation to goal-directed response generation, enabling deployment as a general-purpose assistant.

The reasoning era introduced inference-time compute scaling: models trained with verifiable rewards in mathematics, code, and formal settings learned to allocate additional computation to

difficult problems, producing longer and more deliberate reasoning chains before committing to an answer.

The agentic era optimises models to call tools reliably, maintain coherent state across long contexts, and operate within loops that incorporate external feedback from tool results and environment observations. This is the setting for which the rest of this book provides architectural guidance.

This progression explains why agentic AI is a continuation of the language modelling research programme rather than a departure from it. Agents are the systems layer built on top of the modelling stack; their capabilities and limitations are directly shaped by where the underlying model sits on this trajectory.

3.5 Summary

Frontier models are the capability substrate of agentic systems. The original Transformer introduced attention as a content-dependent communication mechanism; modern frontier decoders extend that foundation with long-context positional representations, efficient attention variants, normalisation improvements, sparse mixture-of-experts layers, post-training alignment, reasoning optimisation, and multimodal input handling. Model selection should be grounded in a capability profile evaluated inside the actual agent workflow, with benchmark results interpreted as system-level measurements rather than intrinsic model properties.

Chapter 4

Cognitive Architecture

The intelligence layer requires a cognitive architecture which defines how the model makes decisions over time.

4.1 The Agent Loop

The defining property of an agentic system is that it pursues a goal across multiple steps rather than producing a single response. This iterative structure is what the cognitive architecture governs, and it is what distinguishes an agent from a prompted language model.

The loop is necessary because the tasks assigned to agents are rarely solvable in a single pass. A software engineering task may require reading failing tests, forming a reasoning trace about the probable cause, modifying code, running tests again, observing a new failure, revising the reasoning, and repeating. A research task may require querying multiple sources, discovering that initial results are insufficient, broadening the search strategy, and synthesising across an expanded set of evidence. In each case the agent must maintain a coherent picture of where it is in the task, what it has learned, and what it should attempt next.

The standard formulation of this loop, introduced by the ReAct framework (Yao et al., 2022), interleaves reasoning and acting in three repeating stages. In the thought stage, the model generates an internal reasoning trace: it analyses the current state of the task and determines what action is most likely to advance it. In the action stage, it executes that action by calling a tool, retrieving context, producing a partial answer, or requesting clarification. In the observation stage, it receives the result of the action and incorporates it into its reasoning before the next thought. This thought-action-observation cycle continues until a termination condition is met. The critical property of the loop is that the agent learns from each observation before deciding on the next step: a tool call that returns an error is an observation that informs the subsequent thought, rather than a terminal failure.

For the loop to function across multiple iterations, the agent must maintain state. State is the accumulated record of what the agent has done and learned: the user's original input, the message history, retrieved documents, tool outputs, the current plan, completed steps, unresolved questions, and accumulated cost.

4.2 Clarification and Autonomy

A recurring decision in the agent loop is whether to seek clarification from the user or to proceed autonomously. The appropriate choice depends on the risk profile of the next action and the

recoverability of an error.

Clarification is warranted when required parameters are absent, when multiple plausible interpretations of the user's intent would lead to materially different actions, when the next action is irreversible, when the cost of an incorrect action is high, or when the applicable permissions are ambiguous. Autonomous continuation is warranted when the next step carries low risk, when the agent can verify the result of its action before committing to it, when the operation is reversible, when the user has explicitly delegated the task, or when missing information can be retrieved safely without user involvement. The design of this decision boundary is a governance choice as much as a technical one: it determines the degree of autonomy the agent exercises and the degree of accountability it shares with the user.

4.3 Stopping Conditions

Because the loop has no natural end when the task is open-ended, the cognitive architecture must specify explicit termination conditions. Without them the agent continues iterating past the point at which the task is complete, consuming budget and potentially degrading a solution that was already correct. Loop termination is itself a model judgment, rather than a deterministic rule. The agent uses the language model to assess whether the current state of the task satisfies the original goal. This means the stopping decision is subject to the same reasoning capabilities and failure modes as any other step in the loop: the model may judge a task complete when it is not, or continue iterating when sufficient evidence already exists.

4.4 Summary

The cognitive architecture controls how the model reasons and acts over time. It structures the agent's execution as a thought-action-observation loop, allocates deliberation budgets, maintains a clear separation between observations and the reasoning trace, governs the boundary between clarification and autonomous action, and enforces termination conditions. The prompt and skill content that informs each reasoning step belongs to the memory and knowledge layer; the cognitive architecture determines how and when that content is used, and for how long the agent continues to pursue the goal.

Part II

The Action Layer

Chapter 5

The Action Layer

The action layer is the boundary at which model decisions become effects on external systems. A model selects an action; the action layer executes it, validates it, records the result, and returns an observation. The design of this layer determines not only what the agent can do, but how reliably and safely it does it.

A common but misleading simplification is to classify tools as deterministic. The relevant distinction is between tools whose outputs are reproducible and whose side effects are predictable, and those where neither holds. A schema validator is fully reproducible. A database lookup is reproducible for a fixed state. A web search varies because rankings change, pages are updated, and snippets are re-extracted. A sub-agent wrapping another model introduces its own reasoning loop and may reach a different conclusion on each invocation. The architecture must treat each tool according to its actual reproducibility and side-effect profile.

5.1 Schemas as Behavioural Contracts

A tool schema is the primary mechanism by which the model learns what a tool does, when it is appropriate to call it, and what constitutes a valid invocation. A well-designed schema gives the tool an unambiguous name, a description that specifies both the conditions under which the tool should be used and those under which it should not, typed arguments with explicit constraints, and a structured output format.

Schema quality has a direct and measurable effect on tool selection fidelity. When two tools have overlapping descriptions, the model resolves the ambiguity arbitrarily. When argument names are vague, parameters are filled incorrectly. When output fields are verbose or unstructured, the model may extract the wrong field or ignore the result entirely. Research has confirmed that poor schema design is a primary driver of tool hallucination, cases where the model selects the wrong tool, supplies malformed parameters, or bypasses the tool entirely and fabricates an output that mimics what the tool would have returned. The schema is therefore a behavioural control: its precision determines the quality of the agent's decisions at the action boundary.

The following example illustrates a well-formed tool definition using the Strands decorator pattern used later in the implementation chapters. The name is unambiguous, the description specifies both the purpose and the condition for use, arguments are typed with explicit constraints, and the return type is structured:

```
from strands import tool
from pydantic import BaseModel, Field
```

```
class JobStatusResult(BaseModel):
    job_id: str
    status: str # "pending" | "running" | "completed" | "failed"
    progress_pct: float = Field(ge=0.0, le=100.0)
    error_message: str | None = None
    summary: str

@tool
def get_job_status(job_id: str) -> JobStatusResult:
    """
    Retrieve the current execution status of a processing job.
    Use this tool after triggering a job with run_job, and before
    retrieving results with get_results. Do not call this tool
    if the job_id is unknown or has not yet been submitted.
    """
    raw = _fetch_job_status(job_id)
    return JobStatusResult(
        job_id=job_id,
        status=raw["status"],
        progress_pct=raw.get("progress", 0.0),
        error_message=raw.get("error"),
        summary=(
            f"Job {job_id} is {raw['status']} "
            f"({raw.get('progress', 0):.0f}% complete)."
        ),
    )
```

Several design choices are worth making explicit. The description instructs the model both when to call the tool and when to refrain. This negative condition is as important as the positive one because it reduces spurious invocations. The return type is a typed Pydantic model rather than a raw dictionary, which enforces the output contract at the Python level and makes the schema visible to the framework. The `summary` field gives the model a pre-formatted string it can incorporate directly into a response without having to synthesise one from the structured fields.

5.2 Read Tools and Write Tools

Tools differ in their side effects, and that difference governs the level of autonomy the agent is granted. Read tools, including search, retrieval, inspection, calculation, carry no persistent consequences and are appropriate for autonomous invocation. Write tools, those that modify external state, send communications, or trigger processes, might require explicit authorisation in the schema and, for consequential operations, a human approval step before execution.

5.3 Agent-as-Tool and the Opacity Problem

When a subtask is sufficiently complex to warrant its own reasoning loop, such as a research task that requires searching multiple sources, a code review that requires inspecting a patch against a set of criteria, a planning task that decomposes a broad request into ordered steps, it can be encapsulated as a sub-agent and exposed to the parent agent as a tool. The parent agent supplies an input and receives an observation; the internal execution is hidden.

This pattern introduces a structural opacity problem. The parent agent cannot inspect the

sub-agent's intermediate decisions unless the sub-agent exposes them through a shared tracing infrastructure. Research on multi-agent failure modes has found that coordination breakdowns, failures at precisely this boundary, represent the largest single category of failure in production multi-agent systems. The consequence is that an agent-as-tool pattern requires nested traces: the sub-agent's execution must appear as a child span within the parent trace, with its tool calls, intermediate reasoning, and output all visible at the same level of detail as any other tool invocation. The diagnostic value of the trace collapses at the delegation boundary when this structure is absent.

5.4 Summary

The action layer exposes external capabilities through typed invocation boundaries. Each tool is a contract: its schema governs how the model selects and invokes it, its read or write classification governs how much autonomy is appropriate, and its output design governs how well the model can reason from the result. When a tool is itself an agent, the contract extends to require that the sub-agent's execution be fully traceable from the parent context. The evaluation chapter builds directly on this structure: scores attached to trajectory spans measure precisely whether each tool was selected correctly, invoked with valid arguments, and its result incorporated appropriately into subsequent reasoning.

Chapter 6

Tools, Function Calling, and MCP

The Model Context Protocol (MCP) is a standardised interface for exposing tools, prompts, and resources to model hosts. It makes the action layer portable: tools can be implemented once and reused across agents, development environments, desktop applications, and cloud deployments.

6.1 The Function-Calling Loop

When a model is given access to tools, each reasoning step follows a structured cycle. The agent framework presents the model with the user request alongside the available tool schemas. The model determines whether the current state of the task requires a tool invocation, and if so emits a structured call containing the tool name and a JSON argument object. The framework validates that call against the schema before passing it to the tool. The tool runs, and the framework returns its output to the model as an observation. The model then either synthesises a final answer or initiates another step.

The model does not require knowledge of the tool's implementation. It requires a precise external contract. This is why schema design, as discussed in the previous chapter, often has greater influence on agent behaviour than prompt prose: a well-specified schema reduces ambiguity before the model reasons, whereas a prompt correction must override a confusion the model has already formed.

6.2 MCP Primitives

The MCP specification defines three principal abstractions.[1] Tools are model-invoked capabilities that interact with external systems, APIs, computations, or stateful actions; each tool is exposed by a server with a name, metadata, and an input schema.[2] Resources are contextual data items exposed by a server and selected by the application or client; a resource may be a file, a document, a database record, or a repository item. Prompts are reusable templates and workflow instructions that clients can discover and retrieve from a server.[3]

This distinction maps directly onto the layered architecture of this book. Tools primarily belong to the action layer. Resources and prompts primarily support the memory and knowledge layer. All three require security boundaries: because any client that can reach an MCP server can invoke its tools, the server must enforce authentication and authorisation independently of the consuming agent.

[1] MCP specification: https://modelcontextprotocol.io/specification/2025-06-18.
[2] MCP tools specification: https://modelcontextprotocol.io/specification/2025-06-18/server/tools.
[3] MCP prompts specification: https://modelcontextprotocol.io/specification/2025-06-18/server/prompts.

6.3 Local Tools and Remote Tools

Tools can run inside the agent process or behind a remote endpoint. A local tool is a plain function co-located with the agent code; it is straightforward to prototype and debug, but its availability is limited to the agent that defines it and its security controls are scoped to the process. A remote tool runs behind an API, a serverless function, an MCP server, or a managed gateway; it introduces network overhead but enables the tool to be shared across multiple agents, governed through identity and policy controls, audited centrally, and scaled independently of the agent.

The practical progression is to develop and stabilise a tool locally, establish its schema and write tests against it, and then promote it to a remote endpoint when it must be shared or brought under organisational governance.

6.4 Validation Before Execution

Tool call validation occurs in two distinct places, and conflating them is a common source of production failures. The first is structural validation: the agent framework checks that the model's emitted arguments conform to the tool's schema before the tool is invoked. When a tool's input type is declared precisely, using typed function signatures and constrained field definitions, the framework can enforce this contract automatically and fail early if the model's output does not conform. The second is authorisation: an independent component verifies that the caller's identity is permitted to invoke the requested capability. This separation ensures that schema conformance and access control are enforced by distinct mechanisms, neither of which depends on the model's own judgment.

6.5 Tool Hallucination

Tool hallucination occurs when the model invokes the wrong tool, constructs arguments that the schema does not support, or reasons as though a tool returned information it did not return. Schema precision is the primary structural mitigation, as established in the previous chapter. At the observability level, the agent's user interface should surface which tools were called and with what arguments, making the model's action choices visible to the user rather than concealing them behind a final answer. At the output level, the model can be instructed to include a structured attestation field in its response, for example, a flag indicating that a consequential action such as submitting a payment was executed, which creates an explicit, checkable link between the model's stated behaviour and the tool invocations recorded in the trace. These mitigations are complementary: schema design reduces the rate of hallucination, observability surfaces instances that occur, and structured attestation makes them detectable in automated evaluation.

6.6 Summary

MCP standardises the tool interface so capabilities are portable across agents and deployment environments. Reliable tool use depends on precise schemas, framework-level validation that is independent of the model, and observability infrastructure that makes every invocation inspectable. The distinction between local and remote tools is a governance boundary as much as an architectural one: the point at which a tool moves to a remote endpoint is the point at which organisational identity, policy, and audit controls become enforceable.

Part III

The Memory and Knowledge Layer

Chapter 7

The Memory and Knowledge Layer

The memory and knowledge layer determines what information is available to the model at the moment of decision. Its function is to construct the context window with the subset of available information that is most relevant and most likely to support a correct and grounded response. Context engineering is the discipline of designing, assembling, and maintaining the information environment available to an agent at inference time, curating the optimal set of tokens during inference, including information beyond the prompt itself.

7.1 Why Memory and Knowledge Form a Single Layer

Memory and knowledge are distinct in origin but unified in function. Memory accumulates through interaction: it records user preferences, past decisions, prior conversation turns, and outcomes. Knowledge is sourced externally: documents, policies, codebases, manuals and databases. Both must pass through the same context engineering process before reaching the model's context window. Context engineering governs this selection, deciding which pieces of memory and which fragments of knowledge to include for a given task, in what form, and at what level of compression.

The two are treated as a single layer because failures in either produce the same downstream effect on grounding: the model reasons from an incomplete or misleading context window. An agent that retrieves irrelevant document chunks, omits a critical policy, or fails to surface a previously stated user preference will produce ungrounded outputs regardless of the model's underlying capability.

7.2 Types of Memory

Memory in agentic systems is usefully classified by its temporal scope and function. Working memory is the immediate task context: the current messages, active plan, tool outputs, and unresolved constraints that the agent is reasoning over in the present turn. Short-term memory spans a session: it preserves conversational history across turns and enables the agent to maintain coherence without re-establishing context on every exchange. Episodic memory records past events across sessions: what the user requested, what the agent did, and what outcome resulted. Semantic memory holds stable extracted facts: user role, project names, known issues, and domain definitions that remain valid across interactions. Preference memory captures standing instructions about tone, format, default tools, and preferred workflows. Procedural memory contains reusable packages of know-how: skills, checklists, playbooks, and workflow templates that the agent can retrieve and apply when a task matches their scope.

7.3 Skills as Procedural Knowledge

A skill occupies a specific position in this taxonomy. It is neither a model weight nor a tool: it is a retrievable unit of procedural knowledge that the system injects into the context window when the current task requires it. A skill may take the form of a domain-specific instruction fragment, a step-by-step checklist for a recurring workflow, a repository development guide, a troubleshooting playbook, or a bundle of tool-use rules paired with expected output formats.

The key property of a skill is that it is selected, versioned, and evaluated independently of the model. A skill whose content is outdated introduces incorrect procedural guidance, and one that is irrelevant to the current task adds noise to the context window and degrades grounding. Skills therefore belong in the knowledge layer and are subject to the same retrieval, quality, and freshness requirements as any other document.

A common representation is a structured Markdown document that pairs machine-readable frontmatter with human-readable procedural content. The frontmatter declares the skill's name, its intended scope, and the tools it is permitted to invoke; the body encodes the step-by-step procedure the agent should follow when the skill is active:

```
---
name: data-pipeline-diagnosis
description: >
  Procedure for diagnosing failures in a data processing pipeline.
  Use when the user reports missing, delayed, or malformed output.
allowed-tools:
  - fetch_job_logs
  - get_job_status
  - inspect_schema
---

## Diagnosis Flow

1. Retrieve the job identifier from the user or from recent job history.
2. Call get_job_status to determine whether the job completed,
   failed, or is still running.
3. If the job failed, call fetch_job_logs and identify the first
   error entry.
4. Call inspect_schema to verify that the input data conforms to
   the expected format.
5. Summarise the root cause and propose a remediation step.
```

This format makes the skill inspectable, diffable, and testable. A new version of the skill can be evaluated against a dataset of past failures before it replaces the previous version in the retrieval index. The agent framework retrieves the skill document when the current task matches its declared scope and injects its content into the context window alongside the user's request.

7.4 Context Compression

Context windows are finite and carry a cost proportional to their length. Compression is the process of reducing the volume of context while preserving the information most relevant to the task at hand. Common techniques include summarising prior conversation turns through memory consolidation, retaining only the highest-ranked retrieval chunks, and converting verbose tool outputs into compact structured fields containing only the decision-relevant values.

Compression introduces its own failure modes. A consolidation step may omit a critical qualification present in the original. A memory extraction step may store a temporary instruction as a permanent preference. A retrieval filter may exclude the document that contains the correct grounding evidence. For this reason, memory and context transformations require the same evaluation discipline as model outputs: they are active interventions in the context engineering pipeline that can introduce errors at any step.

7.5 Memory Write Policy

Selective retention is a design requirement. An agent that stores every piece of information it encounters produces a memory that degrades in utility as it accumulates noise, outdated facts, and low-confidence inferences. A principled write policy stores information when the user has explicitly requested retention, when the fact is stable and reusable across future sessions, when storage improves personalisation or session continuity in a way the user has consented to, and when the information originates from a trusted source. Sensitive information that serves no persistent purpose, temporary context that applies only to the current task, and content inferred with low confidence warrant exclusion from long-term memory.

The system should expose memory to inspection and correction. A user who can query what the agent has retained and modify or delete individual entries has meaningful control over the grounding context the agent will assemble in future sessions.

7.6 Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) is the standard pattern for grounding model responses in external knowledge. It proceeds through four operations. Chunking divides source documents into retrieval units of appropriate granularity, each carrying metadata that identifies its origin, recency, and scope. Embedding maps each chunk into a vector representation that supports semantic similarity search. Retrieval selects the candidate chunks most relevant to the current query, typically by combining vector similarity with keyword or metadata filters. Context assembly arranges the selected chunks into the context window in a form the model can reason from, managing ordering, deduplication, and length constraints.

Each of these operations is a potential source of grounding failure and should be tested independently. A chunking strategy that splits sentences across unit boundaries degrades retrieval precision. An embedding model that has drifted from the domain of the documents produces unreliable similarity scores. A retrieval step that returns the highest-scoring chunks without diversity may miss the document that contains the correct answer. A context assembly step that places the most relevant content at the end of a long context window may cause the model to underweight it due to the well-documented lost-in-the-middle effect. Treating RAG as a pipeline of testable components rather than a monolithic retrieval call is what makes its failure modes diagnosable.

7.7 Summary

The memory and knowledge layer is where skills, context engineering, long-term memory, and retrieval infrastructure belong. Its function is to construct the context window with the right information at the right time. Production agent design is therefore as much a context engineering discipline as a model engineering one: the quality of the context assembled for each task is as consequential as the capability of the model that receives it.

Part IV

The Security and Governance Layer

Chapter 8

The Security and Governance Layer

Agents introduce a category of risk that conversational systems do not. A model that produces an incorrect response misleads a user. An agent that issues an incorrect instruction to a tool can leak sensitive data, send communications, modify records, or deploy broken code to production. Security and governance are therefore foundational properties of the architecture, rather than capabilities added after the core system is built.

8.1 The Attack Surface

The concept of an attack surface describes the set of entry points through which an adversary can influence a system's behaviour. In agentic systems, this surface is substantially wider than in conventional applications. The model's context window is an attack vector: every piece of content that enters it, including retrieved documents, tool outputs, emails, web pages, tickets, and code, can contain instructions that attempt to redirect the model's behaviour. The agent's memory is a further attack vector: content written into long-term memory during one session can influence the model's reasoning in future sessions, a class of attack known as memory poisoning. The tools themselves constitute a third vector: an agent with access to credentials, external APIs, or write operations over persistent storage can be induced to exercise those capabilities in ways that were not authorised.

The principle of least privilege is the foundational response to this expanded attack surface. Each component of the system, including the agent, each tool, and each memory store, should hold only the permissions required to perform its declared function, scoped to the current task, and enforced by the infrastructure rather than assumed from the model's compliance with its system prompt. In agentic contexts, least privilege means limiting an agent's access to only the resources and actions explicitly required to complete its current task, not its general purpose, its potential future tasks, or everything it might conceivably need.

8.2 Identity, Authentication, and Authorisation

Every request in an agentic system should be associated with an identity. That identity may belong to a human user, a service account, a tenant, an organisation, an agent, or a tool. In multi-agent systems, the agent itself carries an identity so that downstream tools can distinguish between a direct human request and an automated action, a distinction that matters for audit, for permission scoping, and for accountability.

Authentication and authorisation operate at two boundaries. Inbound authentication verifies the user or client calling the agent. Outbound authorisation verifies, before any tool executes, whether the authenticated identity holds the permission required to invoke it. When a tool must call a downstream service, the credentials for that call should be scoped, rotated, and injected into the tool execution environment by a secrets manager rather than passed through the model. The model may know that a user holds a particular role; it should have no visibility into OAuth tokens, API keys, or database connection strings.

Permissions should be scoped along multiple axes: user or tenant, data source, tool, action type, environment, time window, budget, and approval requirement. A deployment agent, for example, may be permitted to run tests in any branch, deploy to a staging environment after automated checks pass, and deploy to production only after an authorised engineer has approved the change. Each axis of scoping reduces the consequence of a compromised or misdirected action.

8.3 Prompt Injection

Prompt injection occurs when content from an untrusted source attempts to redirect the model's behaviour. Direct injection arrives in the user's own message. Indirect injection is embedded in content the agent retrieves or processes, such as a web page, a PDF, an email body, a repository file, or a tool output, and is particularly consequential because the user may have no visibility into the injected instruction.

The primary structural mitigations are architectural. Context should be labelled by source and authority level so the model can distinguish between a user instruction and a retrieved passage. Tool access should be scoped to the authenticated user's identity so that an injected instruction cannot invoke a capability the user does not hold. Tool calls should be validated by an independent component rather than trusted on the basis of the model's output alone. Data flows from sensitive sources to external sinks should be explicitly governed. Write operations should require approval for consequential actions. High-risk tools should operate against an allowlist. Every consequential action should be logged with sufficient context to reconstruct the decision that produced it. Infrastructure enforcement is what gives these controls their guarantee.

8.4 Guardrails

Guardrails are validation controls placed at specific points in the agent's execution pipeline, enforcing boundaries that the model itself cannot reliably enforce. An input guardrail inspects the user's request before it reaches the model, blocking unsafe or unsupported inputs. A retrieval guardrail restricts which documents are accessible during context assembly, for example by applying tenant-specific index filters. A tool guardrail validates a tool call before execution, blocking actions such as sending an external communication that contains sensitive data. An output guardrail inspects the model's final response, redacting personal data or flagging policy violations before the response reaches the user. A policy guardrail enforces business rules such as requiring human approval above a cost threshold. A runtime guardrail constrains the execution environment itself, sandboxing shell access and network calls to prevent lateral movement.

The common property across all guardrail types is that they operate independently of the model. The model may propose an action; the guardrail determines whether it executes.

8.5 Data Flow Control

A governed agent architecture tracks the provenance of information from source to sink. A source is where information enters the agent's context: a user profile, a private document, an email, a code repository, a database record, or a web page. A sink is where information leaves the system: a final response, an outbound email, a message, a ticket update, a file write, or an external API call.

Data exfiltration occurs when content from a protected source reaches a sink the user or organisation has not authorised. An agent with read access to internal documents and write access to external communication channels must be governed by explicit data flow rules that an injected instruction cannot override. These rules belong in the infrastructure layer, enforced at the tool boundary.

8.6 Human Approval

For actions that are irreversible or consequential, such as modifying or deleting records, deploying code, submitting a financial or legal document, or sharing internal data outside the organisation, a human approval step is one available control. The appropriate level of human oversight depends on the risk profile of the action and the degree of automation the use case requires. A fully automated pipeline may accept higher risk in exchange for throughput; a governed enterprise deployment may route every write action through an approval queue. The design choice is between automation and accountability, and the right point on that spectrum is determined by the consequences of an unchecked error.

Approval events carry evaluation value regardless of where that point is set. A rejected approval records a case where the agent's proposed action diverged from human judgment. Collected systematically, rejected approvals become labelled examples for the evaluation dataset and regression tests for subsequent agent versions.

8.7 Sandboxing

A sandbox is an environment that replicates production conditions without connecting to production systems. In a staged deployment pipeline, the sandbox stage, typically referred to as gamma, is where the agent is tested against realistic data and realistic tool schemas before any change is promoted to production. The CI/CD pipeline should include a sandbox stack where prompt changes, tool schema updates, and new agent capabilities can be exercised and evaluated in isolation. Capabilities validated in the sandbox environment carry a reproducible evidence record; those that have not been validated there should remain outside the production deployment.

8.8 Governance and Auditability

Governance requires that the organisation can reconstruct, for any past interaction, who invoked the agent, what the model decided, which tools were called, and which policy checks ran. Observability infrastructure that captures every model call, tool invocation, and policy check as a named span with associated metadata is the prerequisite for this. Agent versioning ensures that a trace can be associated with the exact prompt, tool schemas, and skill documents that were active at the time of the interaction, making it possible to reproduce and evaluate historical behaviour as the system evolves.

8.9 Summary

The security and governance layer enforces the boundaries within which the agent operates. The attack surface spans the context window, the memory layer, and the tool interfaces; the principle of least privilege is the foundational response to each. Identity, authentication, and authorisation establish who may act and on what. Guardrails enforce boundaries at every stage of the execution pipeline independently of the model. Prompt injection, data flow control, human approval calibrated to the risk profile of each action, sandboxed validation before promotion, and structured auditability are the core concerns of this layer. Each of them is an infrastructure property enforced independently of the model, and each connects directly to the evaluation and deployment practices covered in subsequent chapters.

Part V

Applications: Building Agents in Production

Chapter 9

Building Agents: Strands and LangGraph

The previous chapters established the conceptual architecture of an agent: layers of intelligence, action, memory, and security. This chapter makes those abstractions concrete by implementing the same assistant twice, once with Strands and once with LangGraph. The tools, the gateway, the skills, and the memory resource are identical in both cases. Only the way the agent loop is assembled differs. Examining both implementations in parallel surfaces what each framework provides and what it leaves to the developer.

The assistant connects to backend capabilities through an MCP gateway secured with AWS Signature Version 4 (SigV4). A `SKILL.md` document encodes the operating procedure. AgentCore Memory keeps track of conversational state and user preferences across sessions. The book follows the order in which components are typically written: tools first, then the agent loop, then memory, then the entrypoint.

9.1 Step 1: Define the tools

Tools are the agent's interface to the world. Before picking a framework, define what the agent can do. Built-in tools are plain Python functions decorated with the `@tool` decorator. The Strands import and the LangGraph import use different packages, but the pattern is identical: a function, a docstring, and an optional return type.

```
# Strands import
from strands import tool

# LangGraph import
from langchain_core.tools import tool

@tool
def get_current_time() -> dict[str, str]:
    """Return the current UTC time."""
    from datetime import datetime, timezone
    return {"utc": datetime.now(timezone.utc).isoformat()}

@tool
def echo(message: str) -> dict[str, str]:
    """Return the message back to the caller. Useful for testing."""
```

```
    return {"response": message}
```

These two tools illustrate the pattern: a decorated function, a docstring that the model uses to decide when to call the tool, and a typed return value. In practice, built-in tools handle lightweight operations that do not require a gateway round-trip, such as simple lookups, formatting, or diagnostic checks. The more capable tools are loaded dynamically from the AgentCore Gateway at startup, as described in the next section.

9.2 Step 2: Load gateway tools over MCP

Gateway tools arrive via the MCP protocol. The two frameworks use different MCP clients, so the gateway loading code diverges here. Strands uses `MCPClient` from the `strands.tools.mcp` package and loads tools synchronously during initialisation. LangGraph uses `MultiServerMCPClient` from `langchain-mcp-adapters` and loads tools asynchronously.

Strands gateway loader.

```
import os
import atexit
from mcp.client.streamable_http import streamablehttp_client
from strands.tools.mcp import MCPClient

def _gateway_urls() -> list[str]:
    raw = os.environ.get("GATEWAY_URL", "")
    return [url.strip() for url in raw.split(",") if url.strip()]

def _make_client(url: str) -> MCPClient:
    region = os.environ.get("GATEWAY_REGION", "us-west-2")
    auth = SigV4HTTPAuth(service=SERVICE_NAME, region=region)
    return MCPClient(
        lambda: streamablehttp_client(
            url=url,
            headers={
                "Accept": "application/json, text/event-stream"
            },
            auth=auth,
        )
    )

class GatewayToolPool:
    def __init__(self) -> None:
        self.clients: list[MCPClient] = []
        self.tools_list: list = []
        self.load_tools()
        atexit.register(self.stop)

    def load_tools(self) -> None:
        urls = _gateway_urls()
        clients = []
        for url in urls:
            client = _make_client(url)
            client.__enter__()
            clients.append(client)
```

```
        tools = []
        for client in clients:
            tools.extend(client.list_tools_sync())
        self.clients = clients
        self.tools_list = tools

    def tools(self) -> list:
        return list(self.tools_list)

    def stop(self) -> None:
        for client in self.clients:
            try:
                client.__exit__(None, None, None)
            except Exception:
                pass
```

`GatewayToolPool` enters each client's context manager on initialisation and registers a cleanup function with `atexit`. The Strands agent holds a reference to this pool and calls `tools()` to get the current list. Because Strands holds persistent MCP client connections, the pool must be kept alive for the duration of the container's lifetime.

LangGraph gateway loader.

```
import os
from langchain_mcp_adapters.client import MultiServerMCPClient

async def load_gateway_tools() -> list:
    urls = _gateway_urls()
    if not urls:
        return []

    region = os.environ.get("GATEWAY_REGION", "us-west-2")
    client_config = {}
    for index, url in enumerate(urls):
        client_config[f"backend_{index}"] = {
            "transport": "http",
            "url": url,
            "auth": SigV4HTTPAuth(
                service=SERVICE_NAME, region=region
            ),
            "headers": {
                "Accept": "application/json, text/event-stream"
            },
        }

    client = MultiServerMCPClient(client_config)
    return await client.get_tools()
```

The LangGraph loader is a coroutine. It awaits `get_tools()` and returns LangChain-compatible tool objects that can be passed directly to `create_react_agent`. Unlike the Strands pool, `MultiServerMCPClient` is stateless by default: each tool invocation creates a fresh MCP session, executes the tool, and closes the session. This means the LangGraph implementation does not require session affinity at the container level, but it incurs a small per-call connection overhead.

9.3 Step 3: Encode operating procedures in SKILL.md

Tools tell the agent *how* to call a capability. A skill tells the agent *when* and *why* to use capabilities in a particular order. The assistant has one skill: a Markdown document that defines the standard operating procedure for the tasks it is responsible for.

The skill document starts with YAML frontmatter that names the skill and declares which gateway tools it is allowed to use:

```
---
name: data-processing-assistant
description: >
  Standard operating procedure for data retrieval,
  job execution, and result review.
allowed-tools:
  - backend___fetch_data
  - backend___run_job
  - backend___check_status
  - backend___get_results
  - backend___create_override
---

## Processing Flow

1. Confirm the input identifier with the user.
2. Fetch the relevant data with fetch_data.
3. Trigger the processing job with run_job.
4. Poll completion status with check_status.
5. Retrieve and summarise results with get_results.
```

The skill registry reads these files and returns metadata for the system prompt and full content for the agent's operating context:

```
from pathlib import Path
from typing import Any

def skill_files(base_dir: Path | None = None) -> list[Path]:
    root = base_dir or Path(__file__).parent
    return sorted((root / "skills").glob("*/SKILL.md"))

def parse_skill_frontmatter(
    skill_file: Path,
    base_dir: Path | None = None,
) -> dict[str, Any]:
    text = skill_file.read_text(encoding="utf-8")
    root = base_dir or Path(__file__).parent
    metadata: dict[str, Any] = {
        "name": skill_file.parent.name,
        "description": "",
        "allowedTools": [],
        "path": str(skill_file.relative_to(root)),
    }
    if not text.startswith("---"):
        return metadata
    end = text.find("\n---", 3)
```

```
    if end == -1:
        return metadata
    current_key = ""
    for raw_line in text[3:end].splitlines():
        line = raw_line.rstrip()
        if not line.strip():
            continue
        if (line.startswith(" - ")
                and current_key == "allowed-tools"):
            metadata["allowedTools"].append(line[4:].strip())
            continue
        if ":" in line and not line.startswith(" "):
            key, value = line.split(":", 1)
            current_key = key.strip()
            if current_key == "name":
                metadata["name"] = value.strip()
            elif current_key == "description":
                metadata["description"] = value.strip()
            elif current_key == "allowed-tools":
                metadata["allowedTools"] = []
    return metadata

def loaded_skills(base_dir: Path | None = None) -> list[dict]:
    root = base_dir or Path(__file__).parent
    return [
        parse_skill_frontmatter(f, root)
        for f in skill_files(root)
    ]
```

The frontmatter parser keeps the logic minimal: it reads YAML keys without a YAML library so the agent package stays lean. Strands consumes the skills directory through a plugin. LangGraph reads the full document content and embeds it in the system prompt.

9.4 Step 4: Build the Strands agent

Strands takes a declarative approach. The developer provides a model, a session manager, plugins, tools, and a system prompt. The framework owns the agent loop: it decides when to call the model, when to invoke tools, and how to stream results back. The agent's code describes *what* the agent is, not *how* its loop runs.

```
import json, os
from pathlib import Path
from strands import Agent, AgentSkills
from bedrock_agentcore.memory.integrations.strands\
    .session_manager import AgentCoreMemorySessionManager

def _model_id() -> str:
    return os.environ.get(
        "MODEL_ID", "us.amazon.nova-2-lite-v1:0"
    )

_agent: Agent | None = None
_gateway_pool: GatewayToolPool | None = None
```

```
def _gateway_tools() -> list:
    global _gateway_pool
    if _gateway_pool is None:
        _gateway_pool = GatewayToolPool()
    tools = _gateway_pool.tools()
    if not tools:
        raise RuntimeError("No gateway tools loaded")
    return tools

def _skills_plugin():
    return AgentSkills(
        skills=str(Path(__file__).parent / "skills")
    )

def _memory_session_manager(
    runtime_session_id: str,
) -> AgentCoreMemorySessionManager:
    actor_id, session_id = _parse_runtime_session_id(
        runtime_session_id
    )
    # Replace with your AgentCore Memory ID and region
    return AgentCoreMemorySessionManager(
        agentcore_memory_config={
            "memory_id": os.environ["MEMORY_ID"],
            "actor_id": actor_id,
            "session_id": session_id,
        },
        region_name=os.environ.get(
            "MEMORY_REGION", "us-west-2"
        ),
    )

def _agent_instance(runtime_session_id: str) -> Agent:
    global _agent
    if _agent is None:
        gateway_tools = _gateway_tools()
        _agent = Agent(
            model=_model_id(),
            session_manager=_memory_session_manager(
                runtime_session_id
            ),
            plugins=[_skills_plugin()],
            tools=[
                get_current_time,
                echo,
                *gateway_tools,
            ],
            system_prompt=(
                "You are a helpful assistant. "
                "Use the available gateway tools to complete "
                "tasks. Follow the loaded skills for standard "
                "operating procedures.\n\n"
                f"Loaded skills: "
```

```
            f"{json.dumps(loaded_skills())}\n\n"
        ),
        callback_handler=None,
    )
return _agent
```

Three things are worth observing here.

First, memory is attached as a *session manager*. The Strands `Agent` receives an `AgentCoreMemorySessionManage` and from that point treats memory as a framework service. It retrieves relevant history, maintains conversational continuity, and writes new interactions to memory as part of the normal agent loop. The application code does not manage memory explicitly.

Second, skills arrive through the `AgentSkills` plugin. The plugin scans the `skills/` directory, parses each `SKILL.md`, and makes its procedures available to the agent without the developer having to embed content manually in a prompt.

Third, the container is pinned to a single session. If the same container receives a request for a different session identifier, the agent raises an error. This is a deliberate trade-off: because Strands holds stateful MCP client connections through the `GatewayToolPool`, reusing a container across sessions would corrupt session state. AgentCore Runtime handles this constraint by routing requests to the correct container.

9.5 Step 5: Build the LangGraph agent

LangGraph makes the agent loop explicit. The developer builds a graph with nodes, edges, and hooks. The prebuilt `create_react_agent` supplies a default ReAct loop: model call, tool call, observation, repeat. However, every part of the loop can be extended, replaced, or instrumented.

Memory components. LangGraph splits memory into two objects. The checkpointer stores graph state per thread so conversations can resume after interruption. The store is a semantic store for long-term memory that supports vector search over past interactions.

```
from langgraph_checkpoint_aws import (
    AgentCoreMemorySaver,
    AgentCoreMemoryStore,
)

def _memory_components():
    memory_id = os.environ["MEMORY_ID"]
    region = os.environ.get("MEMORY_REGION", "us-west-2")
    checkpointer = AgentCoreMemorySaver(
        memory_id, region_name=region,
    )
    store = AgentCoreMemoryStore(
        memory_id=memory_id, region_name=region,
    )
    return checkpointer, store
```

The pre-model hook. The pre-model hook runs before every model call. It is the point where the graph reads long-term memory and injects it into the conversation. LangGraph exposes this hook explicitly because the developer may want to log what was retrieved, filter it, or change

the injection strategy for different graph branches. Strands handles equivalent retrieval inside the framework.

```
import uuid, json
from langchain_core.messages import HumanMessage
from langchain_core.runnables import RunnableConfig
from langgraph.store.base import BaseStore

def _pre_model_hook(
    state: dict,
    config: RunnableConfig,
    *,
    store: BaseStore,
) -> dict:
    configurable = config.get("configurable", {})
    actor_id = str(
        configurable.get("actor_id", "default_user")
    )
    thread_id = str(
        configurable.get("thread_id", "default")
    )
    messages = list(state.get("messages", []))

    latest_human = next(
        (
            m for m in reversed(messages)
            if isinstance(m, HumanMessage)
            or getattr(m, "type", None) == "human"
        ),
        None,
    )

    if latest_human is not None:
        store.put(
            (actor_id, thread_id),
            str(uuid.uuid4()),
            {"message": latest_human},
        )
        memories = store.search(
            (actor_id, thread_id),
            query=str(
                getattr(latest_human, "content", "")
            ),
            limit=5,
        )
        if memories:
            memory_lines = [
                json.dumps(
                    getattr(m, "value", m), default=str
                )
                for m in memories
            ]
            messages.insert(
                0,
```

```
            HumanMessage(
                content=(
                    "Relevant memory context:\n"
                    + "\n".join(memory_lines)
                )
            ),
        )

    return {"llm_input_messages": messages}
```

The hook saves the latest human message to the store and immediately searches the same namespace for relevant prior context. If any is found, it is prepended to the message list before the model sees the conversation. This is explicit: the developer controls exactly when memory is written, what is searched, and how results are formatted before injection.

Assembling the graph. With tools, memory, and hooks in place, the graph is assembled in a single call:

```
from langchain.chat_models import init_chat_model
from langgraph.prebuilt import create_react_agent

_graph = None
_gateway_tools_cache = None

def _build_system_prompt(gateway_tools: list) -> str:
    tool_names = [
        getattr(t, "name", str(t)) for t in gateway_tools
    ]
    return (
        "You are a helpful assistant. "
        "Use the available gateway tools to complete tasks. "
        "Follow the loaded skills for standard operating "
        "procedures.\n\n"
        f"Available gateway tools: {tool_names}\n\n"
        f"Loaded skills: {json.dumps(loaded_skills())}\n\n"
    )

async def _graph_instance():
    global _graph, _gateway_tools_cache
    if _graph is None:
        _gateway_tools_cache = await load_gateway_tools()
        if not _gateway_tools_cache:
            raise RuntimeError("No gateway tools loaded")

        checkpointer, store = _memory_components()
        region = os.environ.get("AWS_REGION", "us-west-2")
        model = init_chat_model(
            _model_id(),
            model_provider="bedrock_converse",
            region_name=region,
        )
        tools = [
            get_current_time,
            echo,
```

```
            *_gateway_tools_cache,
        ]
        _graph = create_react_agent(
            model=model,
            tools=tools,
            checkpointer=checkpointer,
            store=store,
            prompt=_build_system_prompt(_gateway_tools_cache),
            pre_model_hook=_pre_model_hook,
        )
    return _graph
```

The `create_react_agent` call builds a prebuilt ReAct graph: the model decides which tools to call, each tool is executed, and the observation is fed back to the model until it produces a final answer. Skills in LangGraph are embedded in the system prompt: the skill registry reads each `SKILL.md` document and the `_build_system_prompt` function formats them into a single prompt block. There is no plugin; the developer controls the skill injection point directly.

9.6 Step 6: The entrypoint and session identity

Both implementations wrap the agent in a `BedrockAgentCoreApp` and expose an `@app.entrypoint` function. The entrypoint is the interface between the AgentCore Runtime and the agent's domain logic. Both use the same convention: the `runtimeSessionId` encodes both actor and session using a triple-underscore separator.

```
def _parse_runtime_session_id(raw: str) -> tuple[str, str]:
    if "___" in raw:
        actor_id, session_id = raw.split("___", 1)
        return actor_id.strip(), session_id.strip()
    return "default_user", raw.strip()

def _runtime_session_id_from(payload: dict, context) -> str:
    user_id = str(payload.get("user_id") or "").strip()
    session_id = str(payload.get("session_id") or "").strip()
    if user_id and session_id:
        return f"{user_id}___{session_id}"
    context_session_id = getattr(context, "session_id", None)
    if context_session_id:
        return str(context_session_id)
    raise ValueError(
        "Provide user_id + session_id in the payload, "
        "or invoke through AgentCore with a runtimeSessionId"
    )
```

Strands entrypoint. The Strands entrypoint creates or retrieves the agent instance, then calls `stream_async` with the user prompt. The framework handles tool calling, memory updates, and streaming events internally.

```
from bedrock_agentcore import BedrockAgentCoreApp

app = BedrockAgentCoreApp()
```

```
@app.entrypoint
async def invoke(payload: dict, context=None):
    prompt = str(payload.get("prompt") or "").strip()
    if not prompt:
        raise ValueError("prompt is required")
    if prompt.lower() in {"health", "ping", "status"}:
        yield {"message": {"content": [{"text": "ok"}]}}
        return
    runtime_session_id = _runtime_session_id_from(
        payload, context
    )
    agent = _agent_instance(runtime_session_id)
    async for event in agent.stream_async(prompt):
        yield event
```

LangGraph entrypoint. The LangGraph entrypoint awaits the graph, invokes it with the prompt and session configuration, and yields a single response event extracted from the final messages list.

```
from langchain_core.messages import AIMessage

app = BedrockAgentCoreApp()

def _last_ai_text(messages: list) -> str:
    for message in reversed(messages):
        if (
            isinstance(message, AIMessage)
            or getattr(message, "type", None) == "ai"
        ):
            content = getattr(message, "content", "")
            if isinstance(content, str):
                return content
            if isinstance(content, list):
                return "".join(
                    item.get("text", "")
                    if isinstance(item, dict)
                    else str(item)
                    for item in content
                )
    return ""

@app.entrypoint
async def invoke(payload: dict, context=None):
    prompt = str(payload.get("prompt") or "").strip()
    if not prompt:
        raise ValueError("prompt is required")
    if prompt.lower() in {"health", "ping", "status"}:
        yield {"message": {"content": [{"text": "ok"}]}}
        return
    runtime_session_id = _runtime_session_id_from(
        payload, context
    )
    actor_id, session_id = _parse_runtime_session_id(
        runtime_session_id
```

```
    )
    graph = await _graph_instance()
    result = await graph.ainvoke(
        {"messages": [("human", prompt)]},
        config={
            "configurable": {
                "actor_id": actor_id,
                "thread_id": session_id,
            }
        },
    )
    text = _last_ai_text(list(result.get("messages", [])))
    yield {
        "message": {
            "role": "assistant",
            "content": [{"text": text}],
        }
    }
```

The Strands version yields events as they arrive from the framework. The LangGraph version awaits the full graph result and yields one final response. This reflects a deeper difference: Strands is streaming-first by design, while LangGraph's `ainvoke` returns a completed state.

9.7 Step 7: The full picture

With every component defined, the two agents have the same external shape. Both accept a prompt over the AgentCore Runtime boundary. Both authenticate to the same MCP gateway using SigV4. Both load the same skill document. Both store and retrieve memory using the same AgentCore Memory resource. The difference is in what the developer writes.

With Strands, the developer writes an agent *declaration*. The framework owns the loop, the tool dispatch, the memory integration, and the event streaming. The developer chooses model, tools, plugins, and prompt. The control flow is implicit in those choices.

With LangGraph, the developer writes an agent *graph*. The framework provides a graph builder and a default ReAct loop, but the developer can add nodes, attach hooks, add conditional edges, and instrument every transition. Memory integration is explicit: the developer writes the hook that reads memory and decides how to inject it. Skills integration is explicit: the developer reads the skill document and decides how to format it in the prompt.

9.8 Framework comparison

Aspect	**Strands**	**LangGraph**
Agent loop	Owned by the framework; hidden from application code	Explicit graph; developer builds nodes and edges
Gateway auth	`SigV4HTTPAuth` passed to `MCPClient` via `streamablehttp_client`	`SigV4HTTPAuth` passed as `auth` field in `MultiServerMCPClient` config

MCP connection	Persistent connections held in `GatewayToolPool`; tools loaded once at startup	Stateless per tool call; each invocation opens a fresh MCP session
Memory	`AgentCoreMemorySessionManager` as a framework service; retrieval handled internally	`AgentCoreMemorySaver` + `AgentCoreMemoryStore` + explicit pre-model hook
Skills	`AgentSkills` plugin scans the `skills/` directory automatically	Developer reads `SKILL.md` files and embeds content in the system prompt
Streaming	`stream_async()` emits events as the loop runs	`ainvoke()` returns completed state; streaming requires `astream()`
Session affinity	Required: container pinned to a single session because MCP connections are stateful	Not required: stateless MCP sessions allow the container to serve any session
Control flow	Framework decides tool dispatch order	Developer can add conditional edges, interrupts, and custom nodes

9.9 When to choose each framework

The practical question is where complexity lives in the system.

If complexity lives in the tools and operating procedures: the skill is detailed, the gateway exposes many capabilities, the conversation requires careful tool sequencing. Strands keeps the application code compact while the skill documents carry the domain logic. The framework handles the loop; the developer handles the domain.

If complexity lives in the orchestration itself: the agent must coordinate deterministic steps with model-driven steps, expose checkpoints for human approval, branch on domain-specific conditions, or make memory injection visible for evaluation. LangGraph provides explicit structure for representing and testing each part of the trajectory.

Both frameworks integrate with AgentCore Runtime, AgentCore Gateway, and AgentCore Memory. The decision is about where control should live: in the tools and operating procedures, or in the graph itself.

9.10 Summary

Both implementations accept a prompt over the AgentCore Runtime boundary, authenticate to the MCP gateway with SigV4, load tools from the gateway, follow skill-encoded operating procedures, and persist state through AgentCore Memory. The gateway authentication pattern, signing each inbound request with the container's IAM execution role credentials, is the standard approach for any AWS workload calling an IAM-protected service endpoint, and applies equally to both frameworks. What differs is the agent loop. Strands declares an agent and delegates loop control to the framework. LangGraph builds an explicit graph with hooks, memory components, and a system prompt that includes skill content directly. The choice between them is a question of where the developer wants to own control: in the tools and operating procedures, or in the graph itself.

Chapter 10

From local to the cloud: Deploying Agents with AWS AgentCore

Chapter 8 assembled the same assistant in two frameworks, producing two agent implementations that run locally when their environment variables point to real gateway and memory resources. Deployment extends that work into a governed cloud service: it supplies an invocation boundary, an execution role, a memory resource, a governed tool surface, a release mechanism, and the observability needed to diagnose failures in production.

Amazon Bedrock AgentCore provides managed infrastructure for each of these concerns. AWS CDK declares all required resources in code. Examining the CDK stack component by component reveals what each AgentCore service does and why the provisioning order follows the dependencies it does.

10.1 Step 1: Understand what the runtime expects

Before writing infrastructure, it helps to understand the contract the agent code already satisfies. In chapter 8, both agents wrap themselves in a `BedrockAgentCoreApp` and expose exactly one `@app.entrypoint` function:

```
from bedrock_agentcore import BedrockAgentCoreApp

app = BedrockAgentCoreApp()

@app.entrypoint
async def invoke(payload: dict, context=None):
    prompt = str(payload.get("prompt") or "").strip()
    runtime_session_id = _runtime_session_id_from(
        payload, context
    )
    # ... agent logic ...
    yield {
        "message": {
            "role": "assistant",
            "content": [{"text": response_text}],
        }
    }

def main():
```

```
    app.run(host="0.0.0.0", port=8080)
```

`BedrockAgentCoreApp.run()` starts an HTTP server on port 8080. AgentCore Runtime sends invocation requests to this server, supplies the `runtimeSessionId` through the context object, and streams or collects the yielded events. The agent code does not need to know it is running in a container or behind a managed runtime; it only needs to be packaged so that AgentCore can start it.

This separation means the same code can be tested locally by running `python -m agent.app` and passing test payloads directly, and deployed by packaging the `agent/` directory as a Docker image.

10.2 Step 2: Build the tool layer

The runtime does not own the business logic. A well-structured deployment separates tool implementation from agent orchestration by placing domain operations in a Lambda function. The Lambda function is the first resource created in the CDK stack, before the runtime, because the gateway and the runtime both depend on it.

```
import { createToolsResources } from './tools-construct';

const tools = createToolsResources(this, {
  stageName: props.stage.name,
  removalPolicy,
  modelId: DEFAULT_MODEL_ID,
});
```

`createToolsResources` returns a Lambda function and a DynamoDB state table. The Lambda function handles tool invocations from the AgentCore Gateway and any scheduled or event-driven triggers the application requires. The DynamoDB table stores all typed records with a configurable TTL.

This separation gives the deployment a clean ownership model: the agent owns reasoning, the gateway owns tool discovery and protocol, the Lambda owns domain operations, and DynamoDB owns persisted state.

10.3 Step 3: Create the execution role

The agent runtime needs an IAM role that grants it permission to invoke the model, write logs, call the gateway, and access memory. AgentCore assumes this role using a service principal with conditions that prevent confused deputy attacks:

```
const runtimeRole = new iam.Role(this, 'AgentRuntimeRole', {
  assumedBy: new iam.ServicePrincipal(
    'bedrock-agentcore.amazonaws.com'
  ).withConditions({
    StringEquals: {
      'aws:SourceAccount': this.account,
    },
    ArnLike: {
      'aws:SourceArn':
        `arn:aws:bedrock-agentcore:${this.region}:`
        + `${this.account}:runtime/*`,
```

```
    },
  }),
});

runtimeRole.addToPolicy(new iam.PolicyStatement({
  actions: [
    'bedrock:InvokeModel',
    'bedrock:InvokeModelWithResponseStream',
    'cloudwatch:PutMetricData',
    'logs:CreateLogGroup',
    'logs:CreateLogStream',
    'logs:PutLogEvents',
  ],
  resources: ['*'],
}));

tools.toolFunction.grantInvoke(runtimeRole);
dataBucket.grantReadWrite(runtimeRole);
```

The conditions on the trust policy limit which AgentCore resources can assume this role. Without them, any AgentCore runtime in the account could assume the role. The `aws:SourceArn` condition restricts assumption to runtimes in this account and region.

10.4 Step 4: Provision AgentCore Memory

Memory is provisioned as an independent resource before the runtime. Both the Strands and LangGraph implementations receive the memory ID through an environment variable and bind it at startup; they do not create memory resources themselves.

```
import { Memory, MemoryStrategy } from
  '@aws-cdk/aws-bedrock-agentcore-alpha';

const memory = new Memory(this, 'AgentCoreMemory', {
  memoryName: `agent_memory_${props.stage.name}`,
  description: 'Long-term memory for user preferences.',
  expirationDuration: Duration.days(30),
  memoryStrategies: [
    MemoryStrategy.usingUserPreference({
      name: 'user_preferences',
      description: 'Long-term preferences across sessions.',
      namespaces: ['/actors/{actorId}/preferences'],
    }),
  ],
});

memory.grantWrite(runtimeRole);
memory.grantRead(runtimeRole);
```

The memory strategy declares that this resource stores user preferences keyed by actor identifier. AgentCore Memory handles the vector index and retrieval. The agent code uses the memory ID to construct the checkpointer, the store, or the session manager depending on the framework, and the namespace `/actors/{actorId}/preferences` is where both the Strands session manager and the LangGraph pre-model hook write and search.

10.5 Step 5: Create the AgentCore Gateway

The gateway is the MCP endpoint that the agent uses at runtime. It advertises tool schemas, enforces IAM authentication on every tool invocation, and routes calls to the Lambda backend.

```
import {
  Gateway,
  GatewayAuthorizer,
  GatewayProtocol,
  MCPProtocolVersion,
  McpGatewaySearchType,
} from '@aws-cdk/aws-bedrock-agentcore-alpha';

const agentCoreGateway = new Gateway(scope,
  'AgentGateway', {
  gatewayName: `agent-gateway-${stageName}`,
  description: 'MCP gateway for agent tools.',
  authorizerConfiguration: GatewayAuthorizer.usingAwsIam(),
  protocolConfiguration: GatewayProtocol.mcp({
    supportedVersions: [
      MCPProtocolVersion.MCP_2025_06_18,
      MCPProtocolVersion.MCP_2025_03_26,
    ],
    searchType: McpGatewaySearchType.SEMANTIC,
    instructions:
      'Use these tools to retrieve data, trigger processing '
      + 'jobs, inspect results, and record governed decisions.',
  }),
});

agentCoreGateway.addLambdaTarget(
  'AgentToolsTarget', {
  gatewayTargetName: 'agent-tools',
  description: 'Domain tools via Lambda.',
  lambdaFunction: toolFunction,
  toolSchema: ToolSchema.fromInline(AGENT_TOOL_SCHEMA),
});

agentCoreGateway.grantInvoke(runtimeRole);
```

The gateway is not the Lambda function. The Lambda function is a target: it receives invocations from the gateway when a tool is called. The gateway owns the MCP protocol layer, handles tool discovery, and enforces SigV4 authentication. This means the agent code calls the gateway endpoint, not the Lambda directly. The gateway decouples the agent from the underlying implementation so a tool target can be a Lambda, a Fargate service, or any HTTP endpoint without changing the agent code.

The `grantInvoke(runtimeRole)` call allows the runtime container to call the gateway. Without it, the signed requests from the agent would fail with an authorisation error.

The runtime role also needs a specific gateway invocation permission:

```
runtimeRole.addToPolicy(new iam.PolicyStatement({
  actions: ['bedrock-agentcore:InvokeGateway'],
  resources: [
    agentCoreGateway.agentCoreGatewayArn,
```

```
    `${agentCoreGateway.agentCoreGatewayArn}`
    + `/gateway-endpoint/*`,
  ],
}));
```

10.6 Step 6: Package and deploy the Runtime

With the role, memory, and gateway in place, the runtime can be declared. The CDK `Runtime` construct packages the `agent/` directory as a Docker image and registers it with AgentCore:

```
import {
  AgentRuntimeArtifact,
  Runtime,
  RuntimeAuthorizerConfiguration,
  RuntimeNetworkConfiguration,
  ProtocolType,
} from '@aws-cdk/aws-bedrock-agentcore-alpha';
import { ecrAssets } from 'aws-cdk-lib';

const runtime = new Runtime(this, 'AgentRuntime', {
  runtimeName: props.stage.runtimeName,
  executionRole: runtimeRole,
  agentRuntimeArtifact: AgentRuntimeArtifact.fromAsset(
    path.join(__dirname, '..', 'agent'), {
      platform: ecrAssets.Platform.LINUX_ARM64,
    }
  ),
  networkConfiguration:
    RuntimeNetworkConfiguration.usingPublicNetwork(),
  protocolConfiguration: ProtocolType.HTTP,
  authorizerConfiguration:
    RuntimeAuthorizerConfiguration.usingIAM(),
  environmentVariables: {
    STAGE: props.stage.name,
    AWS_REGION: this.region,
    MODEL_ID: DEFAULT_MODEL_ID,
    GATEWAY_URL: agentCoreGateway.agentCoreGatewayUrl,
    GATEWAY_REGION: this.region,
    MEMORY_ID: memory.memoryId,
    MEMORY_REGION: this.region,
  },
});
```

Several details are worth examining.

`AgentRuntimeArtifact.fromAsset` builds a Docker image from the `agent/` directory and pushes it to ECR. The agent directory must contain a `Dockerfile` that installs dependencies and sets the container entry point to the agent application module. CDK handles the build and push during `cdk deploy`.

`environmentVariables` is the bridge between infrastructure and code. The agent reads the gateway URL, memory ID, and model identifier from environment variables. This keeps the same container image portable across stages: a personal stage, a gamma stage, and a production stage all use the same image but receive different environment variables at runtime.

`RuntimeAuthorizerConfiguration.usingIAM()` means that every caller of the runtime must present a valid SigV4 signature. Callers need the `bedrock-agentcore:InvokeAgentRuntime` permission for the runtime ARN.

10.7 Step 7: Add the proxy Lambda for browser clients

Browser applications cannot sign requests with SigV4 directly. The backend solves this with a runtime proxy Lambda that sits between the API Gateway and the AgentCore runtime:

```
const runtimeProxyFn = new lambda.Function(this,
  'RuntimeProxyFunction', {
  runtime: lambda.Runtime.PYTHON_3_11,
  handler: 'index.handler',
  code: lambda.Code.fromAsset(
    path.join(__dirname, '..', 'lambda', 'runtime-proxy')
  ),
  timeout: Duration.seconds(30),
  memorySize: 512,
  environment: {
    ALLOWED_RUNTIME_PATTERN: `^${props.stage.runtimeName}`,
    CORS_ALLOWED_ORIGINS: '*',
  },
});

runtimeProxyFn.addToRolePolicy(new iam.PolicyStatement({
  actions: ['bedrock-agentcore:InvokeAgentRuntime'],
  resources: [
    runtime.agentRuntimeArn,
    `${runtime.agentRuntimeArn}/runtime-endpoint/*`,
  ],
}));

const invokeResource = api.root
  .addResource('runtime')
  .addResource('invoke');

invokeResource.addMethod('POST',
  new apigateway.LambdaIntegration(runtimeProxyFn), {
    authorizationType: apigateway.AuthorizationType.IAM,
  }
);
```

The proxy Lambda accepts a request from the IAM-protected API Gateway endpoint, validates the runtime name against the `ALLOWED_RUNTIME_PATTERN` environment variable, constructs the `runtimeSessionId` from the request, calls `bedrock-agentcore:InvokeAgentRuntime`, parses the streaming response, and returns a JSON envelope to the caller.

The `ALLOWED_RUNTIME_PATTERN` check prevents callers from invoking arbitrary runtimes by constructing a different runtime name. The API Gateway endpoint is protected by IAM, so the browser application authenticates with IAM credentials (typically via Amazon Cognito) and the proxy does the SigV4 signing on its behalf.

This proxy pattern is not convenience wrapping. It is a policy enforcement point: it controls which runtimes can be invoked, how session identifiers are constructed, and how streaming output

is reduced to a response that the browser can parse.

10.8 Step 8: Deploy and verify

The stack is deployed with the CDK CLI:

```
# Install CDK dependencies
npm install

# Deploy to a personal stage
npm run deploy:personal

# Or use CDK directly
npx cdk deploy AgentBackendStack-personal
```

After deployment, CDK outputs the runtime ARN, the API URL, the gateway URL, and the memory ID. These values are the concrete links between the CDK infrastructure and the agent's environment variables.

A minimal smoke test verifies the runtime is alive without invoking any backend tools or the model:

```
import boto3, json, uuid

def invoke_agent(
    runtime_arn: str,
    region: str,
    prompt: str,
) -> str:
    client = boto3.client(
        "bedrock-agentcore", region_name=region
    )
    response = client.invoke_agent_runtime(
        payload=json.dumps(
            {"prompt": prompt}
        ).encode("utf-8"),
        runtimeSessionId=f"testuser___{uuid.uuid4()}",
        agentRuntimeArn=runtime_arn,
    )
    body = response.get("response") or response.get("body")
    return extract_response_text(iter_objects(body))

# Send a health prompt and verify the container is alive
text = invoke_agent(runtime_arn, region, prompt="health")
print(text) # expected: {"status": "ok", ...}
```

The health prompt takes the fast path in both agent implementations: it skips the model, the gateway tools, and the memory layer, and returns a static status object. This makes it safe to call frequently as a liveness check without incurring model costs. For a fuller integration test, send a real task prompt and use Langfuse to capture the trace. The evaluation chapter treats that trace as the foundation of diagnostic scoring.

10.9 What the runtime owns versus what it delegates

The deployment creates a clean separation of ownership across services.

Service	Responsibility
AgentCore Runtime	Packages and runs the agent container; manages invocation lifecycle and session routing
AgentCore Gateway	Exposes backend tools via MCP; enforces SigV4 authentication on every tool call; handles protocol versioning
AgentCore Memory	Stores conversational state, checkpoints, and long-term preferences; provides semantic retrieval
Lambda (tools)	Implements all domain operations; owns business logic and state persistence
DynamoDB	Persists typed records with a configurable TTL
EventBridge	Triggers scheduled or event-driven processing pipelines
API Gateway	Provides browser-facing ingress with IAM authentication and CORS
Lambda (proxy)	Enforces runtime name policy; translates between API Gateway and AgentCore invocation protocol
CloudWatch	Stores structured logs from all Lambda functions; enables query-based diagnostics
Secrets Manager	Stores runtime secrets so they are not baked into the container image

The model is surrounded by conventional cloud services. It does not touch infrastructure directly. When the agent calls a gateway tool, IAM governs the call, the gateway validates the schema, the Lambda runs the domain logic, and DynamoDB stores the result. The agent sees a tool response.

10.10 Strands and LangGraph in the same runtime

Both frameworks deploy to AgentCore using the same CDK stack. The only structural difference is which Python module in the `agent/` directory is set as the entry point in the Dockerfile. The CDK runtime artifact, the IAM role, the gateway, and the memory resource are identical in both cases.

This is the key property of the deployment pattern: it is framework-neutral. The runtime does not care whether the agent is a Strands `Agent` object or a LangGraph `create_react_agent` graph. It cares that the agent exposes an HTTP server on port 8080, accepts a JSON payload, yields events in the expected format, and responds to health checks.

The same memory ID works with the Strands `AgentCoreMemorySessionManager` and with the LangGraph `AgentCoreMemorySaver` and `AgentCoreMemoryStore`. The same gateway URL works with the Strands `GatewayToolPool` and with the LangGraph `MultiServerMCPClient`. The framework choice is an application-layer decision; the deployment layer does not need to change when that decision changes.

10.11 Summary

Deploying an agent to AgentCore follows a fixed order: tool layer first, then IAM role, then memory, then gateway, then runtime, then proxy. Each resource depends on resources created before it. The CDK stack encodes this dependency graph as TypeScript and `cdk deploy` resolves it into a set of CloudFormation change sets.

The agent code stays portable throughout. It reads its gateway URL, memory ID, and model identifier from environment variables. The same code runs locally against a test gateway, on a personal stage, and in production. The deployment layer supplies environment variables and handles authentication; the agent code handles reasoning. That separation is what allows the same assistant to run as a Strands agent or a LangGraph graph without modifying the infrastructure at all.

Chapter 11

Evaluation

Evaluating an agentic system requires choosing, at the outset, what the evaluation is intended to measure. The simplest approach is outcome-based evaluation: given a task, did the agent produce the correct final answer? Its limitation is that a correct answer carries no information about the path taken to reach it. An agent may succeed on a given instance through a sequence of steps that is inefficient, brittle, or unlikely to generalise.

Trajectory-based evaluation addresses this by treating the sequence of decisions as the object of study, not merely its outcome. Rather than scoring the final answer alone, the evaluator inspects the full sequence of tool calls, retrieval steps, and intermediate reasoning: whether the right tools were selected, whether they were invoked with valid arguments, and whether each step was necessary given the state of the task at that point. This approach is more demanding to instrument and score, but it produces a more diagnostic picture of system behaviour, one that can distinguish a robust agent from one that happens to succeed on the test distribution.

11.1 Traces

The practical foundation of both evaluation modes is the trace: a structured, replayable record of a single agent execution. A trace captures the user input, every model call, every tool invocation and its result, any retrieved context, the final answer, per-step latency, and accumulated cost. Without traces, evaluation is limited to comparing inputs and outputs; with them, it becomes possible to isolate the specific decision that caused a failure.

Traces must reflect the hierarchical structure of agent execution. When a parent agent delegates to a sub-agent, that nested execution should appear as a child span within the parent trace. When a tool call fails, the error should be attached to the corresponding observation rather than surfaced only at the top level. This structure is what makes a trace analytically useful rather than merely a log.

11.2 Scores

Evaluation results are represented as scores attached to traces or to individual spans within them. Scores can be produced by automated checks, by human annotators, or by LLM-based judges, and they can target different levels of the execution. At the outcome level, the relevant dimensions include whether the agent satisfied the user's objective, whether factual claims are correct, and whether assertions are grounded in retrieved evidence or tool outputs. At the trajectory level, relevant dimensions include whether the agent selected appropriate tools, whether arguments were

valid and complete, and whether tool results were correctly incorporated into subsequent reasoning. Orthogonal to both are efficiency scores, measuring whether the task was completed within acceptable cost and latency budgets, and safety scores, which assess whether the agent respected applicable permissions and constraints.

11.3 Datasets and Experiments

Langfuse supports input/expected-output datasets as the baseline format for offline evaluation. As the system is deployed and production traces accumulate, the dataset can be versioned and progressively enriched: traces flagged during online audit, where the agent failed or behaved unexpectedly, become new labelled examples in the next dataset version. This mechanism ensures that the evaluation suite tracks the actual failure modes encountered in production, and that fixes introduced in a new agent version are verified against the exact conditions that exposed the original defect.

11.4 Stratified Sampling with Unbalanced Weights

Production traffic is rarely uniform. In a typical deployment, the majority of conversations receive no explicit feedback, a small fraction receive a positive signal such as a thumbs-up, and a smaller fraction still receive a negative signal. Annotating a simple random sample of this traffic would leave the most informative cases, especially those where the agent visibly failed, severely underrepresented.

Stratified sampling addresses this by partitioning traffic into buckets and allocating annotation capacity to each bucket independently. In the feedback example, the natural partition is:

Bucket	**Meaning**
$h = \uparrow$	Thumbs up
$h = \downarrow$	Thumbs down
$h = \emptyset$	No feedback

Let W_h denote the true production share of bucket h, and let n_h denote the number of examples annotated within that bucket. For each bucket, the observed defect rate is:

$$\hat{p}_h = \frac{\text{number of defective examples in bucket } h}{n_h}$$

The overall production defect rate is then recovered by re-weighting each bucket estimate by its production share:

$$\hat{p}_{\text{overall}} = \sum_h W_h \hat{p}_h$$

This is the stratified sampling correction: because annotation overrepresents certain buckets, each bucket-level estimate must be scaled back to its true weight before aggregation. The approximate variance of this estimator is:

$$\text{Var}(\hat{p}_{\text{overall}}) = \sum_h W_h^2 \frac{\hat{p}_h(1-\hat{p}_h)}{n_h}$$

and the resulting 95% confidence interval is:

$$\hat{p}_{\text{overall}} \pm 1.96 \sqrt{\sum_h W_h^2 \frac{\hat{p}_h(1-\hat{p}_h)}{n_h}}$$

A practical consequence of this formula is that uncertainty in the overall estimate is dominated by the largest production bucket. If the no-feedback population represents 98.5% of traffic, then $W_\emptyset^2$ is an order of magnitude larger than the weights of the other buckets, and the no-feedback variance term will dominate the confidence interval regardless of how many thumbs-down examples are annotated. Annotating high-signal buckets such as thumbs-down is valuable for diagnosing failure modes, but it does not substitute for adequate coverage of the no-feedback population when the goal is to estimate overall production performance reliably.

Numerical example

Suppose production traffic is distributed as follows:

Bucket	**Prod. share** W_h	**Annotated** n_h	**Defect rate** $\hat{p}_h$
Thumbs up	1.0%	200	2%
Thumbs down	0.5%	500	65%
No feedback	98.5%	2 000	8%

The weighted defect estimate is:

$$\hat{p}_{\text{overall}} = 0.010 \times 0.02 + 0.005 \times 0.65 + 0.985 \times 0.08 = 0.0823$$

The standard error is:

$$SE = \sqrt{0.010^2 \cdot \frac{0.02 \times 0.98}{200} + 0.005^2 \cdot \frac{0.65 \times 0.35}{500} + 0.985^2 \cdot \frac{0.08 \times 0.92}{2000}} \approx 0.00598$$

The 95% confidence interval is therefore [7.05%, 9.40%]. The no-feedback term contributes virtually all of the variance, illustrating that increasing the thumbs-down annotation budget would narrow the interval only marginally.

11.5 LLM-as-a-Judge

Some evaluation criteria resist rule-based scoring: helpfulness, clarity, groundedness, and policy compliance each require a degree of interpretive judgment. LLM-based judges can operationalise these criteria at scale, but they introduce systematic biases that must be accounted for, including verbosity bias, position bias, and a tendency toward self-preference when the judge and the agent under evaluation share the same base model. Mitigation requires explicit, criterion-anchored rubrics; eliciting a chain of evidence before requesting a numerical score; enforcing structured output; running at low temperature; and calibrating judge scores against human annotations on a held-out subset.

LLM judges should not displace deterministic checks where deterministic checks are applicable. Conformance to a JSON schema should be verified by a schema validator. Functional correctness of generated code should be verified by executing tests. The role of an LLM judge is to handle the residual criteria that deterministic methods cannot reach.

11.6 Failure Taxonomy

The diagnostic value of traces is substantially higher when failures are categorised consistently. A working taxonomy for agentic systems covers three failure modes: incorrect tool usage, which encompasses both wrong tool selection and malformed input parameters; incorrect output, which covers both hallucination and wrong format; and guardrails violation. Labelling failures according to such a taxonomy transforms a collection of individual errors into a distribution of failure modes, which in turn makes it possible to prioritise improvements and measure whether interventions are effective.

11.7 Offline and Online Evaluation

Offline evaluation runs a fixed dataset before deployment and functions as a gate: a promotion is blocked if quality falls below a defined threshold. Online evaluation samples live production traffic after deployment and functions as a monitor: it detects distribution shift, novel user behaviour, tool failures, and model drift that fixed datasets do not anticipate. The two modes are complementary and a mature system requires both. Offline evaluation gives deployment confidence; online evaluation gives operational visibility. Human annotation periodically calibrates the automated scoring pipeline, and production trace mining continuously supplies new candidate items for the offline dataset, closing the loop between the two modes.

11.8 Summary

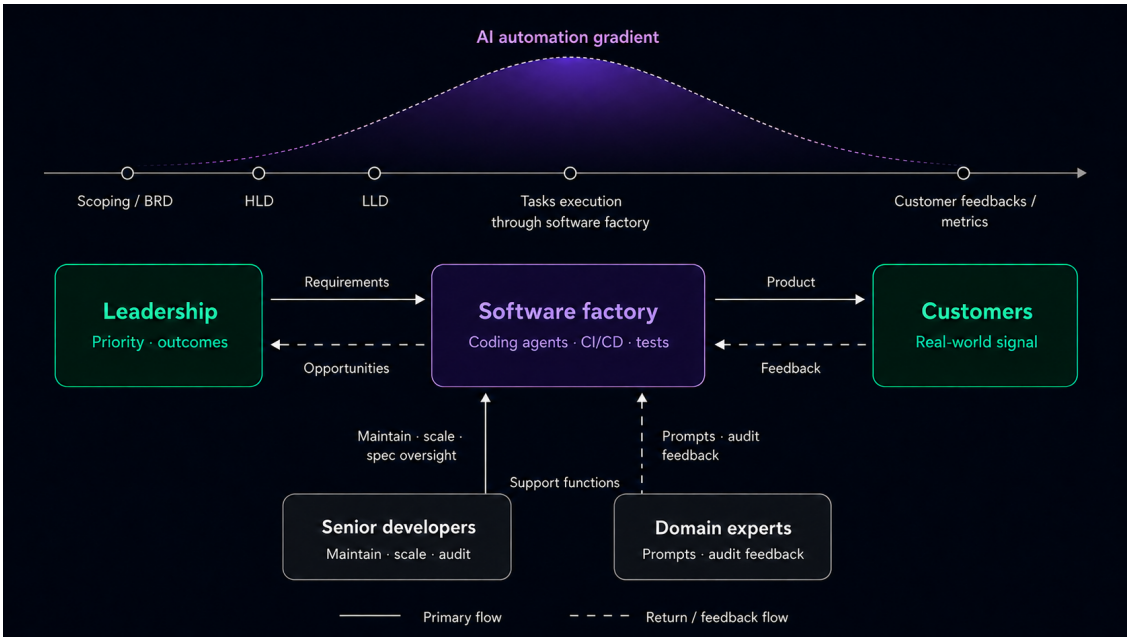
Agent evaluation is not a single procedure but a system of interlocking practices: choosing between outcome and trajectory evaluation, instrumenting traces, defining scored dimensions, maintaining a versioned dataset, deploying LLM judges with appropriate mitigations, and operating both offline and online pipelines. The connecting principle is that every production failure should become a labelled example, every labelled example should become an evaluation, and every evaluation should become a deployment gate.

Part VI

Building an AI-First Company

Chapter 12

Building an AI-First Organization



The automation gradient

In modern enterprise product development, the operating model can be represented as one continuous loop:

business problem identification → BRD → HLD → LLD → tasks → implementation → feedback → new tasks or updated business problem/design.

In an AI-first company, automation follows an *AI gradient*. The center of the loop is the most automatable because it is more technical, structured, and repeatable. The edges of the loop remain the most human-intensive because they require judgment, ambiguity resolution, and accountability.

At the beginning of the loop, humans identify the business problem, set the priority, and define the expected outcome. This is where human expertise is hardest to replace: identifying the right problem before it is fully specified, inferring what is unspoken, anticipating risks before they appear, and recognizing opportunities that are not yet obvious from the data.

As work moves toward the center of the loop, AI can take on more of the implementation work. It can help convert a business requirement document into a high-level design, a high-level design into

a low-level design, and a low-level design into executable tasks. Coding agents[1] can then implement the required software changes through the software factory. In this context, the software factory is the set of skills, tools, tests, and CI/CD mechanisms that allow coding agents to generate software compliant with company requirements.

The system leverages feedback signals such as SME audits through sampling, customer feedback, and actuals data. These signals are converted into metrics that compare current performance against the ideal state. Based on the gap, the AI agent decides the next action. If the gap is small or localized, it creates a new task. If the gap suggests a deeper issue in the business logic or system design, it brings the problem back to the leadership and proposes updates to the business problem, BRD, HLD, or LLD.

12.1 Minimum Architecture

The minimum architecture requires three components.

The first component is the *software factory*. This is the engine that converts structured tasks into production-grade software. It uses coding agents, tests, CI/CD, and deployment gates to generate changes that are compliant with company requirements.

The second component is a small set of *MCP interfaces* that allow humans and AI to communicate through the systems where work already happens. In a first version, these interfaces can connect to Asana, Outlook/Mail, and Slack. Asana acts as the execution surface, while Outlook and Slack act as communication surfaces.

The third component is the *knowledge base*. This stores the key documents and context required by the AI system, including BRDs, HLDs, LLDs, system context, ownership information, and relevant operating assumptions.

12.2 Organizational Structure

This operating model requires a corresponding shift in organizational structure. The AI-first company does not look like a traditional corporate pyramid, with authority concentrated at the top and execution capacity at the base. It looks more like a net, with the software factory at the center.

On the left sits leadership, which sends business requirements into the factory and receives back a continuous signal of improvement opportunities and new initiatives surfaced from data. On the right sit customers, whose usage feedback flows directly back into the factory as the primary signal for improving code and reprioritizing work. The horizontal axis, from business intent to customer reality, is the main value-delivery path, and the factory is the engine that runs along it.

Below the factory sit its two support functions. Senior developers maintain and scale the factory itself, ensuring that coding agents can generate software compliant with company standards. They oversee the conversion of business requirements into specifications and audit the factory's output. Domain experts are engaged selectively, depending on the application. Where involved, they contribute by authoring prompts that encode specialized knowledge and by providing feedback during the audit cycle. Both roles exist to make the factory more reliable and more capable, not to execute tasks directly.

[1] For implementation references on agent loops, see Claude Agent SDK: Agent Loop and OpenAI: Unrolling the Codex Agent Loop.

12.3 Competitive Advantage

As foundation models commoditize, so does software development itself. When any company can convert a business requirement into production code through a software factory, the code is no longer the moat. Competitive advantage shifts to what the factory cannot generate on its own.

Three sources of durable advantage remain.

The first is *proprietary data*. The more unique and hard to replicate a company's data is, the more the factory can be trained, grounded, and evaluated against signals that competitors do not have access to. Data collected through sensors, APIs, operational systems, and direct customer interaction becomes a compounding asset. Data collected closer to the physical world, at higher frequency, and across longer time horizons compounds in value faster than any generic software capability.

The second is *grounding in the physical world*. Companies with sensors, machinery, logistics networks, industrial facilities, and physical operations possess something that software-native competitors cannot easily acquire: a direct feedback loop between model decisions and real-world outcomes. As AI agents become orchestrators of complex workflows, they will increasingly consume real-world services as tools, such as routing a shipment, dispatching a vehicle, or operating a piece of equipment. A company that exposes its physical operations through a well-designed tool interface becomes a node in the global agentic infrastructure. The harder that service is to replicate, the more durable that position is.

The third is the *capacity to experiment*. The software factory lowers the cost of each iteration, but the ability to run many experiments in parallel, absorb the cost of failure, and redeploy learnings quickly is itself a function of organizational discipline and investment appetite. Companies that build rigorous test-fail-learn pipelines, and treat each failure as a labelled example rather than a sunk cost, accumulate a compounding advantage in model and system quality that slower-moving competitors cannot close through technology alone.

Part VII

Mathematical Foundations

Chapter 13

From Hilbert Spaces to σ-Algebras: The Transformer Explained at a Higher Level of Mathematical Abstraction

13.1 Introduction

A transformer is the composition of two maps of different mathematical kinds. The first, from the input tokens to the last hidden state, is an operator between finite-dimensional Hilbert spaces: nonlinear as a whole, but assembled from linear maps, row-wise normalizations and softmax-weighted averages. The second, the output layer, is a Markov kernel from the last hidden state to the vocabulary: the softmax assigns to each hidden state a probability measure on the σ-algebra of token events, and the next token is a random variable with that law. This chapter develops both maps for the original Transformer of 2017.

The chapter has three main sections.

- **Mathematical Tools** (Section 13.2) collects the tools: Hilbert and Banach spaces, linear and nonlinear operators, spectral theory, weak derivatives and Sobolev spaces, measure-theoretic probability built on the σ-algebra of events, the softmax as a Gibbs measure, Fourier analysis, finite-precision arithmetic, information theory, and approximation theory.
- **The Original Transformer** (Section 13.3) describes the 2017 architecture as its authors presented it, with light notation.
- **The Transformer as Operators and Measures** (Section 13.4) writes the 2017 model as a composition of operators, then shows how its last layer defines a probability measure on the σ-algebra of token events and how tokens are sampled from it.

The chapter concerns what a transformer is: a parametrized family of measures $\{P_\theta\}$. How one member of the family is chosen, through pretraining, post-training, alignment and reinforcement learning, is a separate subject that this chapter does not treat.

The map from architecture to mathematics. Each component of the model corresponds to one object of Section 13.2. Table 13.1 lists, for each component of the architecture, the mathematical object and the subsection that develops it. Every subsection of Section 13.2 appears on the right.

Table 13.1: Each component of the 2017 Transformer, the mathematical object it corresponds to, and the subsection that develops it.

Component	Mathematical object	Developed in
Hidden states $X \in \mathbb{R}^{n \times d}$	Finite-dimensional Hilbert space, Frobenius inner product	§13.2.1
Layer stacking with $\circ$	Composition of operators, Lipschitz constants, chain rule	§13.2.2
Token embedding	Linear map into the Hilbert space	§13.2.1
Positional encoding	Orthogonal rotations, Fourier shift theorem	§13.2.3, §13.2.8
Attention scores $QK^\top$	Bilinear form, SVD, rank $\leq d_k$, Eckart–Young	§13.2.3
Attention softmax	Row-stochastic matrix, Markov kernel over positions	§13.2.3, §13.2.6
Cross-attention over the memory	Integral operator with the exponential kernel, compact operators, Mercer (in projected coordinates)	§13.2.4
Residual + LayerNorm	Id + G; projection theorem; compact set of states	§13.2.1, §13.2.2
Feed-forward ReLU	Piecewise-linear map, weak derivative, $W^{1,\infty}$	§13.2.5
Output logits	Linear functionals, Riesz representers	§13.2.1
Output softmax	Probability measure on the σ-algebra $2^{\mathcal{V}}$, Gibbs measure	§13.2.6, §13.2.7
Temperature, top-k, nucleus	Gibbs family, conditioning on an event	§13.2.7
Sampling a token	Random variable with that law, Gumbel-max	§13.2.6, §13.2.7
Causal mask	Predictability along a filtration, Doob–Dynkin	§13.2.6
Autoregressive loop	Measure on sequences, Ionescu-Tulcea, π-λ uniqueness	§13.2.6
End-of-sequence token	Stopping time, absorbing Markov chain	§13.2.6
Training target, label smoothing	Conditional expectation as projection, cross-entropy and KL	§13.2.6, §13.2.10
Floating-point evaluation	Finite precision, underflow of the measure	§13.2.9
Expressivity of the architecture	Stone–Weierstrass, universality	§13.2.11

The same correspondence, read in the other direction, organizes Section 13.4.

Notation. Vectors are rows when they are rows of a matrix $X \in \mathbb{R}^{n \times d}$, one row per token. In bilinear forms such as $x^\top B\, y$, and in matrix–vector products such as Wh, the vectors involved are read as columns.

13.2 Mathematical Tools

This section builds every tool that Section 13.4 uses, in the order in which they depend on each other. Finite-dimensional linear algebra comes first, then its infinite-dimensional extensions, then

measure-theoretic probability, then the analytic tools for positions, precision, information and approximation.

13.2.1 Vector spaces, inner products, and Hilbert spaces

A real vector space V is a set closed under addition and scalar multiplication. An inner product $\langle \cdot, \cdot \rangle$ on V equips it with geometry: it induces a norm $\|x\| = \sqrt{\langle x, x \rangle}$, a distance $\|x - y\|$, and a notion of orthogonality, $\langle x, y \rangle = 0$. A Hilbert space is an inner-product space that is complete: every Cauchy sequence converges to a point of the space.

Completeness is a nontrivial condition in infinite dimensions but holds automatically in finite dimensions. Every finite-dimensional inner-product space is a Hilbert space, isometrically isomorphic to $\mathbb{R}^d$ with the Euclidean inner product. The space of matrices $\mathbb{R}^{n \times d}$ with the Frobenius inner product is a Hilbert space of dimension nd:

$$\langle X, Y \rangle_F = \operatorname{tr}(X^\top Y) = \sum_{i,j} X_{ij} Y_{ij}.$$

In finite dimensions all norms are equivalent: for any two norms there are constants $c, C > 0$ with $c\|x\|_a \le \|x\|_b \le C\|x\|_a$. Convergence and continuity therefore do not depend on the norm, although the constants can depend strongly on the dimension.

The projection theorem. Let M be a closed subspace of a Hilbert space H (in finite dimensions every subspace is closed). Every $x \in H$ decomposes uniquely as

$$x = P_M x + (x - P_M x), \qquad P_M x \in M, \quad x - P_M x \perp M,$$

and $P_M x$ is the unique point of M closest to x. The map P_M is linear, idempotent and self-adjoint. Least squares, and in §13.2.6 conditional expectation, are instances of this theorem.

The Riesz representation theorem. Every continuous linear functional f on a Hilbert space H has the form $f(x) = \langle x, y_f \rangle$ for a unique $y_f \in H$, and $\|f\| = \|y_f\|$. In $\mathbb{R}^d$ this says that every linear functional is $x \mapsto u \cdot x$ for a unique vector u, its representer. A vector can therefore be read in two ways: as a point of the space, or as the functional it represents.

13.2.2 Operators: linear, nonlinear, and piecewise

An operator is a map $T \colon V \to W$ between vector spaces. It is linear if $T(\alpha x + \beta y) = \alpha T(x) + \beta T(y)$. Between finite-dimensional spaces, every linear operator is represented by a matrix once bases are fixed, and is bounded, with operator norm

$$\|T\| = \sup_{\|x\|=1} \|Tx\|,$$

whose value depends on the norms chosen on V and W.

Nonlinear operators are studied through weaker structures.

- **Lipschitz continuity.** T is L-Lipschitz if $\|T(x) - T(y)\| \le L\|x - y\|$ for all x, y. It is locally Lipschitz if such a bound holds on every bounded set, with a constant that may grow with the set.
- **Differentiability.** T is Fréchet differentiable at x if there is a linear operator $DT(x)$, the Jacobian in coordinates, such that $T(x + h) = T(x) + DT(x)h + o(\|h\|)$. Differentiation replaces a nonlinear operator locally by a linear one.

Both properties compose well: Lipschitz constants multiply, $L(T \circ S) \le L(T) \cdot L(S)$, and derivatives obey the chain rule, $D(T \circ S)(x) = DT(S(x))\, DS(x)$. Backpropagation is the chain rule applied, in reverse order, to a long composition.

Piecewise-smooth maps. A map $T\colon \mathbb{R}^m \to \mathbb{R}^p$ is piecewise smooth if there are finitely many disjoint open sets $\Omega_1, \dots, \Omega_r$ whose closures cover $\mathbb{R}^m$, T is smooth on each Ω_j, and the boundary set Γ has Lebesgue measure zero. Such a map is differentiable almost everywhere. It need not be continuous across Γ, and when continuous it need not be Lipschitz.

Two examples delimit the class. The ReLU function $x \mapsto \max(0, x)$ is piecewise linear, continuous and 1-Lipschitz. The top-k selection operator, which returns the index set $S_k(s)$ of the k largest entries of a score vector $s \in \mathbb{R}^N$, is piecewise constant and jumps where two scores tie. A map of the form

$$F(x) = \sum_{i \in S_k(s(x))} g_i(x)$$

is smooth away from the tie set $\{x : s_i(x) = s_j(x) \text{ for some } i \neq j\}$ and generally discontinuous on it. The tie set has measure zero whenever the score differences $s_i - s_j$ are non-constant real-analytic functions, because the zero set of a non-constant real-analytic function has Lebesgue measure zero. This is the real-variable counterpart of the identity theorem for holomorphic functions, and it applies to networks built from analytic activations such as exp and GELU.

Rademacher's theorem states that every locally Lipschitz map is differentiable almost everywhere. It covers ReLU networks but not networks containing hard selections.

Absolute continuity and the fundamental theorem of calculus. Almost-everywhere differentiability does not recover a function from its derivative. On an interval, the identity $f(b) - f(a) = \int_a^b f'$ holds exactly when f is absolutely continuous, and every Lipschitz function is absolutely continuous. The Cantor–Vitali function is the standard counterexample: it is continuous and non-decreasing from 0 to 1, yet $f' = 0$ almost everywhere. Its almost-everywhere derivative carries no information about how the function changes. Step functions fail more severely: they are discontinuous, and their derivative is zero wherever it exists. For quantizers and hard selection, gradients computed almost everywhere are therefore uninformative, and training requires surrogate gradients.

13.2.3 Spectral theory in finite dimensions

The spectral theorem. A linear operator A on a real inner-product space is self-adjoint if $\langle Ax, y\rangle = \langle x, Ay\rangle$. Every self-adjoint operator on a finite-dimensional space has an orthonormal basis of eigenvectors with real eigenvalues:

$$A = Q\Lambda Q^\top, \qquad Q^\top Q = I, \quad \Lambda = \operatorname{diag}(\lambda_1, \dots, \lambda_d).$$

The theorem requires linearity and symmetry and says nothing about nonlinear maps.

Orthogonal operators. A matrix Q with $Q^\top Q = I$ is an isometry, $\|Qx\| = \|x\|$, and all its singular values equal 1. A block-diagonal matrix of 2×2 rotations R_θ is the basic example. Rotations satisfy $R_\alpha R_\beta = R_{\alpha+\beta}$: composing them adds angles, so they represent translation linearly. Identifying $\mathbb{R}^2$ with $\mathbb{C}$, R_θ is multiplication by $e^{i\theta}$ (§13.2.8).

The singular value decomposition. Any $W \in \mathbb{R}^{m\times n}$ of rank r factors as

$$W = U\Sigma V^\top = \sum_{i=1}^{r} \sigma_i\, u_i v_i^\top, \qquad \sigma_1 \ge \cdots \ge \sigma_r > 0,$$

with orthonormal singular vectors. The SVD can be derived from the spectral theorem applied to $W^\top W$, whose nonzero eigenvalues are the σ_i^2. The largest singular value is the Euclidean operator norm, $\|W\|_2 = \sigma_1$, and hence the Euclidean Lipschitz constant of $x \mapsto Wx$.

Low-rank approximation. By the Eckart–Young–Mirsky theorem, truncating the SVD after k terms gives the best rank-k approximation in both the spectral and Frobenius norms:

$$\|W - W_k\|_2 = \sigma_{k+1}, \qquad \|W - W_k\|_F = \Big(\sum_{i>k} \sigma_i^2\Big)^{1/2}.$$

Any product $W = BA$ with $B \in \mathbb{R}^{m\times k}$ and $A \in \mathbb{R}^{k\times n}$ has rank at most k, so factoring through a k-dimensional space imposes a rank bound by construction.

The polar decomposition. For $W \in \mathbb{R}^{m\times n}$ with $m \geq n$, the thin SVD gives

$$W = (UV^\top)\,(V\Sigma V^\top),$$

the product of a matrix with orthonormal columns and a positive semidefinite matrix. The polar factor $UV^\top$ is the nearest matrix with orthonormal columns in the Frobenius norm: it keeps the directions of W and sets every singular value to 1. Newton–Schulz iterations compute it using matrix products only.

Stochastic matrices. $P \in \mathbb{R}^{n\times m}$ is row-stochastic if $P \geq 0$ entrywise and every row sums to 1. For any $V \in \mathbb{R}^{m\times d}$, each row of PV is a convex combination of rows of V, and Hölder's inequality with exponents 1 and ∞ gives, for every column j,

$$|(PV)_{ij}| \leq \sum_k P_{ik}\,|V_{kj}| \leq \max_k |V_{kj}|.$$

A row-stochastic matrix is therefore non-expansive in the maximum norm, $\|P\|_{\infty\to\infty} = 1$. It need not be non-expansive in the Euclidean norm: the $n \times n$ matrix whose rows all equal $e_1^\top$ has $\|P\|_2 = \sqrt{n}$.

Doubly stochastic matrices. A square matrix B is doubly stochastic if both B and $B^\top$ are row-stochastic. By the Birkhoff–von Neumann theorem, the doubly stochastic matrices form the convex hull of the permutation matrices (the Birkhoff polytope). Two consequences follow:

- every doubly stochastic matrix satisfies $\|B\|_2 \leq 1$, since each permutation matrix has norm 1 and the norm is convex;
- the polytope is closed under multiplication, so products of arbitrarily many doubly stochastic matrices remain non-expansive.

By the Sinkhorn–Knopp theorem, for every matrix A with strictly positive entries there are positive diagonal matrices D_1, D_2, unique up to a scalar, such that $D_1 A D_2$ is doubly stochastic. Alternating row and column normalization converges to it; stopping after finitely many steps gives an approximately doubly stochastic matrix.

13.2.4 Banach spaces, compact operators, and the spectral theorem

Two later topics need infinite dimensions. Approximation theory (§13.2.11) works in spaces of functions that are generally not Hilbert spaces, and the kernel view of attention (Section 13.4) describes attention as an integral operator, whose structure is governed by the spectral theory of compact operators.

Banach spaces. A normed space is a Banach space if it is complete. Every Hilbert space is Banach; a norm comes from an inner product exactly when it satisfies the parallelogram law $\|x+y\|^2 + \|x-y\|^2 = 2\|x\|^2 + 2\|y\|^2$. Two examples recur:

- $C(K)$, the continuous functions on a compact $K \subset \mathbb{R}^d$ with the sup norm $\|f\|_\infty = \max_K |f|$: Banach but not Hilbert, and the natural setting for uniform approximation;
- $L^p(\mu)$ for $1 \le p \le \infty$, with $\|f\|_p = \left(\int |f|^p \, d\mu\right)^{1/p}$: Hilbert only for $p = 2$. Hölder's inequality $\|fg\|_1 \le \|f\|_p \|g\|_q$ $(1/p + 1/q = 1)$ and Minkowski's inequality $\|f+g\|_p \le \|f\|_p + \|g\|_p$ are its basic tools.

Two finite-dimensional facts fail in infinite dimensions.

1. **Norms are no longer equivalent.** On $C([0,1])$ there are sequences (f_n) with $\|f_n\|_1$ constant and $\|f_n\|_\infty \to \infty$. A claim such as "networks approximate f to accuracy ε" is incomplete until the norm is named.
2. **Bounded sets need not have compact closure.** By Riesz's lemma, the closed unit ball is compact if and only if the space is finite-dimensional. Compactness, automatic in $\mathbb{R}^d$ by Heine–Borel, becomes a special property.

Bounded and compact operators. A linear operator $T\colon X \to Y$ is bounded if $\|T\| = \sup_{\|x\| \le 1} \|Tx\|$ is finite, which is equivalent to continuity; when Y is Banach, the bounded operators $B(X,Y)$ form a Banach space. T is compact if it maps bounded sets to sets with compact closure. Finite-rank operators are compact, and on Hilbert spaces the compact operators are exactly the operator-norm limits of finite-rank operators.

The main example is the integral operator. For a finite Borel measure μ on a compact K and a kernel $\kappa \in L^2(\mu \otimes \mu)$,

$$(T_\kappa f)(u) = \int_K \kappa(u,v)\, f(v)\, d\mu(v)$$

is a Hilbert–Schmidt, hence compact, operator on $L^2(\mu)$. Every continuous kernel on a compact set defines one.

The spectral theorem for compact self-adjoint operators. Let T be compact and self-adjoint on a separable Hilbert space H. There are an orthonormal sequence (e_i) and real eigenvalues λ_i with $|\lambda_1| \ge |\lambda_2| \ge \cdots$ and $\lambda_i \to 0$ such that

$$Tf = \sum_i \lambda_i \, \langle f, e_i \rangle \, e_i \qquad \text{for all } f \in H,$$

and each nonzero eigenvalue has a finite-dimensional eigenspace. This is the infinite-dimensional analogue of $A = Q\Lambda Q^\top$, with the additional constraint that the eigenvalues can accumulate only at zero. For compact operators that are not self-adjoint, the Schmidt decomposition $Tf = \sum s_i \langle f, v_i \rangle u_i$ plays the role of the SVD, with singular values s_i decreasing to zero. Eckart–Young carries over: truncating after k terms is optimal, with error s_{k+1} in operator norm. The decay of the singular values measures how compressible the operator is.

Mercer's theorem. A symmetric kernel κ is positive definite if every matrix $[\kappa(u_i, u_j)]$ built from finitely many points is positive semidefinite. Let K be compact, μ a finite Borel measure with support K, and κ continuous, symmetric and positive definite. Then T_κ is compact, self-adjoint and positive, and

$$\kappa(u,v) = \sum_i \lambda_i \, \phi_i(u) \, \phi_i(v), \qquad \lambda_i \geq 0,$$

with absolute and uniform convergence on $K \times K$. The kernel is an inner product of feature vectors $\Phi(u) = (\sqrt{\lambda_i}\, \phi_i(u))_i$ in ℓ^2, and these features span its reproducing kernel Hilbert space (RKHS).

The exponential kernel $\kappa(q,k) = e^{\langle q,k \rangle}$ is positive definite: its Taylor series is a sum, with nonnegative coefficients, of powers of the linear kernel, and powers of a positive-definite kernel remain positive definite by the Schur product theorem. On a compact set Mercer's theorem applies to it. Because the kernel is analytic, its eigenvalues decay rapidly; the decay slows as the domain grows, because the kernel then varies over a wider range of scales. The precise rate depends on the dimension and the domain radius; see Steinwart and Christmann for quantitative statements.

13.2.5 Weak derivatives and Sobolev spaces

The states of a transformer are vectors in finite-dimensional spaces, so Sobolev spaces do not describe the state space. They are the right language for two other questions: how regular the network map itself is, and how regular a target function must be for networks to approximate it at a given rate (§13.2.11).

Weak derivatives and distributions. Let $\Omega \subset \mathbb{R}^m$ be open and f locally integrable. A locally integrable g is the weak partial derivative $\partial_i f$ if

$$\int_\Omega f \, \partial_i \varphi \, dx = -\int_\Omega g \, \varphi \, dx \qquad \text{for all } \varphi \in C_c^\infty(\Omega).$$

More generally, a distribution is a continuous linear functional T on the test functions $C_c^\infty(\Omega)$, and its derivative is defined by $\langle \partial_i T, \varphi \rangle = -\langle T, \partial_i \varphi \rangle$. Every locally integrable function defines a regular distribution $\varphi \mapsto \int f\varphi$, and every distribution has derivatives of all orders.

The model example is ReLU. Its weak derivative is the Heaviside step $H = \mathbf{1}_{(0,\infty)}$, and its second distributional derivative is the Dirac delta δ_0, the functional $\varphi \mapsto \varphi(0)$. A network built from ReLU and linear maps is piecewise linear, so its classical Hessian is zero almost everywhere; its curvature is concentrated, as a measure, on the piecewise-linear hypersurfaces (hyperplanes for a single layer, bent hyperplanes for deeper networks) where some unit switches.

Sobolev spaces. For $k \in \mathbb{N}$ and $1 \leq p \leq \infty$,

$$W^{k,p}(\Omega) = \{ f \in L^p(\Omega) : \partial^\alpha f \in L^p(\Omega) \text{ for all } |\alpha| \leq k \},$$

with weak derivatives, is a Banach space; $H^k = W^{k,2}$ is a Hilbert space. Two facts connect this to §13.2.2.

- **Lipschitz functions are $W^{1,\infty}$.** On $\mathbb{R}^m$, a function is locally Lipschitz exactly when it belongs to $W^{1,\infty}_{\text{loc}}$, and its weak gradient coincides with the classical gradient that Rademacher's theorem provides almost everywhere. Dense attention, smooth activations, ReLU and normalization (smooth everywhere for $\epsilon > 0$) all produce network maps in this class.
- **Jumps leave the Sobolev scale.** A function that jumps across a hypersurface is not in $W^{1,1}_{\text{loc}}$. It is locally of bounded variation: its distributional derivative is a measure with an absolutely continuous part (the almost-everywhere gradient) and a singular jump part concentrated on the hypersurface. Hard top-k selection, discrete routing and quantizers produce maps of this kind. The almost-everywhere gradient sees only the first part and misses the jumps entirely, which is the precise form of the warning at the end of §13.2.2.

13.2.6 Measure, probability, and the σ-algebra of events

This section builds the probabilistic language of the chapter. Its central object is the σ-algebra: the collection of events to which probabilities can be consistently assigned.

σ-algebras and events. A σ-algebra $\mathcal{F}$ on a set Ω is a collection of subsets that contains Ω and is closed under complements and countable unions. The pair $(\Omega, \mathcal{F})$ is a measurable space, and its elements are events. The axioms guarantee that "not", "or" and "and", applied to countably many events, again yield events. For any collection $\mathcal{C}$ of subsets, $\sigma(\mathcal{C})$ denotes the smallest σ-algebra containing $\mathcal{C}$, the σ-algebra it generates.

The token σ-algebra. When Ω is finite or countable, the power set 2^Ω is the natural σ-algebra. For a finite vocabulary $\mathcal{V}$, the space $(\mathcal{V}, 2^{\mathcal{V}})$ has $2^{|\mathcal{V}|}$ events; its atoms are the singletons $\{v\}$, and every event is a finite union of atoms. Events have direct linguistic readings: "the next token is punctuation", "the next token begins a word", "the next token lies in a given top-k set".

Borel σ-algebras. On $\mathbb{R}^d$ the power set is not a usable σ-algebra, since Lebesgue measure cannot be extended to all subsets. The standard choice is the Borel σ-algebra $\mathcal{B}(\mathbb{R}^d)$, generated by the open sets. Continuous and piecewise-smooth functions are Borel measurable. Hidden states, and continuous inputs such as image embeddings, live in measurable spaces of this kind.

Measures, conditioning, and construction. A probability measure P on $(\Omega, \mathcal{F})$ is countably additive with $P(\Omega) = 1$. On a countable space it is determined by its values on atoms, $P(S) = \sum_{\omega \in S} p(\omega)$. For an event S with $P(S) > 0$, conditioning gives the measure $P(A \mid S) = P(A \cap S)/P(S)$, supported on S. A sub-probability measure satisfies $P(\Omega) \leq 1$; adjoining a cemetery point ∂ that carries the missing mass turns it into a probability measure.

Measures on large σ-algebras are built from their values on simple generating classes. Carathéodory's construction extends a countably additive set function from an algebra to the σ-algebra it generates. Uniqueness comes from Dynkin's π-λ theorem: a π-system is a class closed under finite intersections, and two probability measures that agree on a π-system agree on the σ-algebra it generates.

Random variables, information, and Doob–Dynkin. A map $X\colon (\Omega, \mathcal{F}) \to (E, \mathcal{E})$ is measurable if $X^{-1}(B) \in \mathcal{F}$ for every $B \in \mathcal{E}$; on a probability space it is a random variable. Its law $P^X(B) = P(X \in B)$ is the pushforward of P. Each random variable generates $\sigma(X) = \{X^{-1}(B) : B \in \mathcal{E}\}$, the events decided by the value of X: its information. The Doob–Dynkin lemma makes this precise. If Y takes values in $\mathbb{R}^d$ or a finite set, then Y is $\sigma(X)$-measurable if and only if $Y = f(X)$ for some measurable f. "Y depends only on X" and "Y is measurable with respect to the information in X" are the same statement.

Densities and Radon–Nikodym. A measure ν is absolutely continuous with respect to μ, written $\nu \ll \mu$, if $\mu(A) = 0$ implies $\nu(A) = 0$. For σ-finite measures, the Radon–Nikodym theorem gives a density $d\nu/d\mu \geq 0$ with $\nu(A) = \int_A (d\nu/d\mu)\, d\mu$ for every A. On a finite space the density is simply the ratio of masses, $\nu(\{v\})/\mu(\{v\})$. Likelihood ratios and the Kullback–Leibler divergence (§13.2.10) are defined through Radon–Nikodym derivatives.

Conditional expectation as a projection. Let $\mathcal{G} \subset \mathcal{F}$ be a sub-σ-algebra. For $X \in L^1(P)$, the conditional expectation $\mathbb{E}[X \mid \mathcal{G}]$ is the almost surely unique $\mathcal{G}$-measurable random variable with

$$\int_A \mathbb{E}[X \mid \mathcal{G}]\, dP = \int_A X\, dP \qquad \text{for all } A \in \mathcal{G}.$$

For $X \in L^2$, the $\mathcal{G}$-measurable square-integrable variables form a closed subspace $L^2(\Omega, \mathcal{G}, P)$ of the Hilbert space $L^2(\Omega, \mathcal{F}, P)$, and $\mathbb{E}[X \mid \mathcal{G}]$ is exactly the orthogonal projection of X onto it (§13.2.1). It is therefore the best predictor of X in mean square among all functions of the information in $\mathcal{G}$. Conditional probabilities are the case $X = 1_A$: $P(A \mid \mathcal{G}) = \mathbb{E}[1_A \mid \mathcal{G}]$.

Kernels. A Markov kernel from $(E, \mathcal{E})$ to $(\Omega, \mathcal{F})$ assigns to each $e \in E$ a probability measure $K(e, \cdot)$ on $\mathcal{F}$, measurably in e. It is a measure that varies with an input, the formal object behind every conditional distribution in this chapter. A row-stochastic matrix is exactly a Markov kernel between finite sets; if $K(e, \Omega) \leq 1$ is allowed, the kernel is sub-Markov.

Sequence space, cylinder sets, and filtrations. Let $\Omega = \mathcal{V}^{\mathbb{N}}$ be the space of infinite sequences over a finite alphabet, with product σ-algebra. The cylinder sets

$$C(x_1, \ldots, x_t) = \{\omega \in \Omega : \omega_1 = x_1, \ldots, \omega_t = x_t\}$$

fix a prefix. For each t, $\mathcal{F}_t = \sigma(\omega_1, \ldots, \omega_t)$ is generated by the cylinders of length t and contains every event decided by the first t symbols. The σ-algebras are nested, $\mathcal{F}_0 = \{\emptyset, \Omega\} \subset \mathcal{F}_1 \subset \mathcal{F}_2 \subset \cdots$, and form a filtration; $\mathcal{F}_\infty = \sigma(\bigcup_t \mathcal{F}_t)$ is the product σ-algebra. The cylinders together with $\emptyset$ form a π-system generating $\mathcal{F}_\infty$, so by the π-λ theorem a probability measure on sequence space is uniquely determined by the probabilities of all prefixes.

Adaptedness, predictability, and stopping times. A process (Y_t) is adapted if each Y_t is $\mathcal{F}_t$-measurable, and predictable if each Y_t is $\mathcal{F}_{t-1}$-measurable: determined strictly by the past. By Doob–Dynkin, predictability means $Y_t = f_t(\omega_1, \ldots, \omega_{t-1})$. A stopping time is a random time τ with $\{\tau = t\} \in \mathcal{F}_t$ for every t: whether to stop is decidable from the information available at that time.

Construction of sequence measures. The Ionescu-Tulcea theorem states that a sequence of Markov kernels $p(\cdot \mid x_{<t})$ from the past to the next symbol determines a unique probability measure on $(\Omega, \mathcal{F}_\infty)$. On cylinders it satisfies the chain rule:

$$P(C(x_1, \ldots, x_t)) = \prod_{s=1}^{t} p(x_s \mid x_{<s}).$$

When generation starts from a prompt, the construction applies conditionally on it.

Markov chains and absorption. Such a process is a time-homogeneous Markov chain on the countable space of finite prefixes (a prefix records its own length, so no separate time index is needed): each step appends one symbol according to a kernel that depends on the current prefix. A state is absorbing if the chain, once there, never leaves. Absorption problems ask for the probability that the chain is eventually absorbed; Section 13.4 identifies this probability with termination of generation.

13.2.7 The softmax and Gibbs measures

For $z \in \mathbb{R}^K$ and a temperature $\tau > 0$, the softmax

$$\operatorname{softmax}_\tau(z)_k = \frac{e^{z_k/\tau}}{\sum_j e^{z_j/\tau}}$$

maps any real vector to a strictly positive probability vector, and hence to a probability measure on $(\{1, \ldots, K\}, 2^{\{1,\ldots,K\}})$ that charges every atom.

- **Shift invariance.** $\operatorname{softmax}_\tau(z + c\mathbf{1}) = \operatorname{softmax}_\tau(z)$ for every constant c. Only logit differences matter; the logits live naturally in the quotient $\mathbb{R}^K/\mathbb{R}\mathbf{1}$.
- **Gradient and curvature.** The softmax is the gradient of the convex function $\tau \log \sum_j e^{z_j/\tau}$. At $\tau = 1$ its Jacobian, which is also the Hessian of log-sum-exp, is $\operatorname{diag}(p) - pp^\top$. This is the covariance matrix of the one-hot vector drawn from p, and the Fisher information of the categorical family in its natural parameters. It is symmetric positive semidefinite with $\mathbf{1}$ in its null space, and $\operatorname{softmax}_\tau$ is $(1/\tau)$-Lipschitz in the Euclidean norm.

- **Variational characterization.** The softmax is the Gibbs distribution:

$$\operatorname{softmax}_\tau(z) = \arg\max_{p\in\Delta^{K-1}} \{\langle p, z\rangle + \tau H(p)\}, \qquad H(p) = -\sum_k p_k \log p_k.$$

This is convex duality: the Fenchel conjugate of log-sum-exp is negative entropy on the simplex and $+\infty$ off it. Temperature weights entropy against expected score. As $\tau \to 0$ the measure converges weakly to the uniform measure on the argmax set, a Dirac mass when the maximizer is unique; as $\tau \to \infty$ it tends to the uniform measure.

- **Masking and restriction.** Logits set to $-\infty$ receive probability zero. Applying the softmax only to the logits indexed by a set S produces exactly the conditional measure $P(\cdot \mid S)$ of §13.2.6: restricting the softmax to a subset is conditioning on that event.
- **Extra denominator mass.** Adding $b > 0$ to the denominator yields a sub-probability vector, equivalently a probability vector on $\{1, \dots, K\} \cup \{\partial\}$ with mass $b/(b + \sum_j e^{z_j})$ on the cemetery point.
- **Sampling as a measurable map of noise.** If $G_1, \dots, G_K$ are independent standard Gumbel variables, then $\arg\max_k(z_k + G_k)$ has law $\operatorname{softmax}_1(z)$ (the Gumbel-max trick).
- **Stable computation.** Shift invariance justifies computing the softmax after subtracting $\max_k z_k$, which prevents overflow. Partial results over blocks combine associatively: a block summarized by (m, s), with m its maximum logit and s the sum of $e^{z_k - m}$, merges with another as $m = \max(m_1, m_2)$, $s = s_1 e^{m_1 - m} + s_2 e^{m_2 - m}$. This online softmax lets attention be computed block by block without storing the full score matrix.

13.2.8 Fourier analysis and positional structure

Complex exponentials and rotations. Identify $\mathbb{R}^2$ with $\mathbb{C}$. The rotation R_θ becomes multiplication by $e^{i\theta}$, and the map $m \mapsto e^{im\theta}$ turns translation of an integer position m into multiplication: $e^{i(m+k)\theta} = e^{ik\theta} e^{im\theta}$. This identity underlies both Fourier analysis and positional encodings.

Fourier series. The functions $e_k(x) = e^{ikx}/\sqrt{2\pi}$, $k \in \mathbb{Z}$, form an orthonormal basis of $L^2(-\pi, \pi)$. Every $f \in L^2$ expands as $f = \sum \langle f, e_k\rangle e_k$, and Parseval's identity $\|f\|^2 = \sum |\langle f, e_k\rangle|^2$ says the coefficient map is an isometry onto ℓ^2.

The Fourier transform. For $f \in L^1(\mathbb{R}^d)$, $\hat{f}(\xi) = \int f(x) e^{-ix\cdot\xi}\, dx$. It satisfies the Riemann–Lebesgue lemma, inversion for suitable f, and, by Plancherel's theorem, extends to a multiple of a unitary operator on L^2. Two properties are used below:

$$\widehat{f(\cdot - a)}(\xi) = e^{-ia\cdot\xi}\, \hat{f}(\xi), \qquad \widehat{f * g} = \hat{f}\, \hat{g}.$$

The shift theorem says translation acts on frequencies as a phase rotation. The convolution theorem says every bounded translation-invariant linear operator on L^2, which is a convolution (in general with a distribution), is diagonalized by the Fourier transform.

Phases and their ambiguity. The argument of a complex number is defined only modulo 2π; the complex logarithm is multivalued, and a branch must be chosen. For positional structure this means that the phase $m\theta$ determines the position m only modulo $2\pi/\theta$. A single frequency cannot distinguish positions that differ by a full period (aliasing). Several frequencies in geometric progression resolve positions over a range set by the longest period. Incommensurate frequencies never alias exactly on the integers, but beyond that range distinct positions become numerically hard to tell apart.

13.2.9 Finite-precision arithmetic

Digital hardware does not compute in $\mathbb{R}$. A floating-point format defines a finite set $\mathbb{F} \subset \mathbb{R} \cup \{\pm\infty\}$, and rounding is a map $\mathrm{fl}\colon \mathbb{R} \to \mathbb{F}$ sending each real number to a nearby element. Within the normal range $|\mathrm{fl}(x) - x| \le u|x|$, where the unit roundoff u is set by the number of mantissa bits.

Low-precision formats combine a small grid with a shared scale: a block of b values is stored as $x_i \approx s \cdot q_i$, each q_i from the grid and one scale s per block. The 4-bit E2M1 format has magnitudes $\{0, 0.5, 1, 1.5, 2, 3, 4, 6\}$ and a sign bit, so the largest value in a block is $6 \cdot s_{\max}$, where $s_{\max}$ is the largest representable scale.

A quantizer $Q = \mathrm{fl}$ has three properties:

- it is idempotent, $Q \circ Q = Q$;
- it is piecewise constant on each coordinate, hence nonlinear and locally of bounded variation;
- its derivative vanishes except on the finite set of rounding thresholds, where it jumps.

By §13.2.2 and §13.2.5, its almost-everywhere derivative carries no information, so training through a quantizer needs a surrogate gradient, commonly the straight-through estimator, which treats DQ as the identity.

Norm bounds determine whether a format can overflow. If normalization bounds the Euclidean norm of a state by R, every coordinate is bounded by R, and orthogonal maps such as rotations preserve that bound. A format whose largest magnitude exceeds R cannot overflow on those states. The normalized states lie on a compact set, so continuous maps of them are bounded (Weierstrass) and uniformly continuous (Heine–Cantor).

The same remarks apply to probabilities. The exact softmax is strictly positive, but in floating point small probabilities can underflow to zero: the computed measure can assign zero mass to atoms that the idealized measure charges.

13.2.10 Information theory

Let p and q be probability vectors on a finite set. The entropy, cross-entropy and Kullback–Leibler divergence are

$$H(p) = -\sum_k p_k \log p_k, \qquad H(p, q) = -\sum_k p_k \log q_k, \qquad \mathrm{KL}(p \,\|\, q) = \sum_k p_k \log \frac{p_k}{q_k}.$$

They satisfy $H(p, q) = H(p) + \mathrm{KL}(p \,\|\, q)$. The divergence is the expected log Radon–Nikodym derivative, $\mathrm{KL}(P \,\|\, Q) = \mathbb{E}_P[\log dP/dQ]$, defined when $P \ll Q$ and $+\infty$ otherwise. By Gibbs' inequality $\mathrm{KL}(p \,\|\, q) \ge 0$, with equality exactly when $p = q$; minimizing cross-entropy in q therefore recovers p. KL is not symmetric: when q ranges over a restricted family (for example unimodal q and multimodal p), minimizing $\mathrm{KL}(p \,\|\, q)$ over q spreads q over all of p's mass, while minimizing $\mathrm{KL}(q \,\|\, p)$ concentrates q on a mode of p.

For measures on sequence space built by the chain rule, divergences decompose along the filtration:

$$\mathrm{KL}(P \,\|\, Q) = \sum_{t \ge 1} \mathbb{E}_P\Big[\mathrm{KL}\big(P(\cdot \mid x_{<t}) \,\|\, Q(\cdot \mid x_{<t})\big)\Big].$$

The perplexity of a model q on a text $x_1, \dots, x_T$ is $\exp\big(-(1/T)\sum_t \log q(x_t \mid \text{earlier tokens})\big)$: the exponential of the average per-token cross-entropy. A perplexity of k means the model is, on average, as uncertain as a uniform choice among k tokens.

13.2.11 Approximation theory

Approximation theory asks which functions a class of networks can represent, and at what cost. Its results are existence statements: they say nothing about whether training finds the approximating network, or whether it generalizes.

Density in $C(K)$ and Stone–Weierstrass. Let $K \subset \mathbb{R}^d$ be compact. The Stone–Weierstrass theorem states that a subalgebra of $C(K)$ that contains the constants and separates points is dense in the sup norm. The span of the exponentials $x \mapsto e^{w\cdot x+b}$ is such an algebra, since $e^{a\cdot x} \cdot e^{b\cdot x} = e^{(a+b)\cdot x}$: networks with exponential units are therefore dense in $C(K)$. The exponential is the nonlinearity inside softmax attention.

Universality of shallow networks. For standard activations σ the span of $\{\sigma(w \cdot x + b)\}$ is not an algebra, and other arguments are needed. Cybenko (1989) proved density for sigmoidal activations by duality: if the span were not dense, Hahn–Banach would give a nonzero continuous functional vanishing on it, which by the Riesz–Markov theorem is a nonzero signed measure, and the sigmoidal structure forces that measure to be zero. Leshno, Lin, Pinkus and Schocken (1993) gave the sharp statement: for continuous σ, the span is dense in $C(K)$ if and only if σ is not a polynomial. Density can also be stated in $L^p(\mu)$; by §13.2.4 the two statements are different claims.

Rates and regularity classes. Density gives no rate. Rates require assuming the target is regular, and the regularity scales are those of §13.2.5.

- **Sobolev classes and the curse of dimensionality.** For f in the unit ball of $W^{n,\infty}([0,1]^d)$, Yarotsky (2017) shows that deep ReLU networks reach sup-norm accuracy ε with $O(\varepsilon^{-d/n}\log(1/\varepsilon))$ weights and $O(\log(1/\varepsilon))$ depth. The exponent d/n expresses the curse of dimensionality: the required size grows as a power of $1/\varepsilon$ whose exponent is proportional to the dimension and inversely proportional to the smoothness.
- **Optimality.** Continuous nonlinear widths (DeVore, Howard and Micchelli) show that $\varepsilon^{-d/n}$ cannot be beaten when the parameters depend continuously on f. Very deep networks with discontinuous parameter selection can beat it (Yarotsky, 2018); this is unrelated to the "super-convergence" of learning-rate schedules (Smith and Topin).
- **Besov classes.** Besov spaces capture smoothness that varies across the domain, such as functions that are smooth except near local singularities. Deep networks adapt to this inhomogeneity, while linear methods, which approximate from a fixed subspace, provably cannot (Suzuki, 2019).
- **Barron classes.** For functions with a finite first Fourier moment, $\int |\omega|\,|\hat{f}(\omega)|\,d\omega < \infty$, shallow networks with N units achieve L^2 error $O(C_f N^{-1/2})$ (Barron, 1993). The exponent does not depend on d, although the constant C_f can. The proof is Fourier-analytic (§13.2.8).

The hypotheses of these results differ in detail and should be taken from the original sources; DeVore, Hanin and Petrova, *Neural Network Approximation* (Acta Numerica, 2021), is a single reference covering most of them.

13.3 The Original Transformer

The Transformer of Vaswani et al., *Attention Is All You Need* (2017), replaced recurrence and convolution in sequence transduction with attention alone. This section describes it as the paper does, with light notation; Section 13.4 analyzes it.

13.3.1 The task and the overall architecture

The model was built for machine translation. An encoder maps a source sequence of symbol representations to continuous representations $z = (z_1, \dots, z_n)$. A decoder then generates the target sequence one symbol at a time, auto-regressively, consuming its previous outputs as additional input.

Both stacks have $N = 6$ identical layers of width $d_{\text{model}} = 512$.

- **Encoder layer:** multi-head self-attention, then a position-wise feed-forward network.
- **Decoder layer:** masked multi-head self-attention, then multi-head attention over the encoder output, then a feed-forward network.

Each sub-layer is wrapped in a residual connection followed by layer normalization, so its output is $\text{LayerNorm}(x + \text{Sublayer}(x))$. This is the post-norm arrangement.

Table 13.2: Data flow in the original encoder–decoder Transformer.

Stage	Feeds into
Source tokens	Encoder: 6 layers self-attention + FFN
Encoder: 6 layers self-attention + FFN	Cross-attention in every decoder layer
Previous outputs	Decoder: 6 layers masked self-attention + FFN
Cross-attention in every decoder layer	Decoder: 6 layers masked self-attention + FFN
Decoder: 6 layers masked self-attention + FFN	Linear + softmax next-token probabilities

The encoder reads the whole source; the decoder reads its own past outputs and, through cross-attention, the encoder's output.

13.3.2 Scaled dot-product attention

Queries and keys have dimension d_k and values dimension d_v. Packing them into matrices Q, K, V, the attention function is

$$\text{Attention}(Q, K, V) = \text{softmax}\Big(\frac{QK^\top}{\sqrt{d_k}}\Big)V.$$

Each output is a weighted sum of values, with weights given by a compatibility function of the query and the keys. The authors explain the factor $1/\sqrt{d_k}$ as a remedy: they suspected that for large d_k the dot products grow large and push the softmax into regions with very small gradients.

13.3.3 Multi-head attention

Instead of one attention function at full width, the model projects queries, keys and values h times with learned matrices and runs attention in parallel:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)\, W^O, \qquad \text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V).$$

The base model uses $h = 8$ heads with $d_k = d_v = 64$. The authors' motivation is that multiple heads let the model attend jointly to information from different representation subspaces. In their ablations, reducing the key size d_k hurt quality, which they read as a sign that determining compatibility is not easy.

13.3.4 Three uses of attention

1. **Encoder self-attention:** queries, keys and values all come from the previous encoder layer, and every position attends to every position.
2. **Masked decoder self-attention:** each position attends only to positions up to and including itself. Illegal connections are masked by setting their softmax inputs to $-\infty$. Combined with shifting the output embeddings by one position, this makes the prediction at position i depend only on outputs before i.
3. **Encoder–decoder attention:** queries come from the previous decoder layer; keys and values come from the encoder output, so every decoder position can attend over the whole input.

13.3.5 Feed-forward networks, embeddings, and the output layer

Each layer applies the same two-layer network to every position separately, $\mathrm{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$, with inner dimension 2048. Input and output tokens are embedded by learned matrices, and a linear map followed by a softmax converts the decoder output into next-token probabilities. The two embedding matrices and the pre-softmax linear map share one weight matrix; in the embedding layers it is multiplied by $\sqrt{d_{\text{model}}}$.

13.3.6 Positional encoding

With no recurrence or convolution, order must be injected explicitly. The model adds sinusoidal encodings to the input embeddings:

$$PE_{(pos,\,2i)} = \sin\!\left(pos/10000^{2i/d_{model}}\right), \qquad PE_{(pos,\,2i+1)} = \cos\!\left(pos/10000^{2i/d_{model}}\right).$$

The wavelengths form a geometric progression from 2π to $10000 \cdot 2\pi$. The authors chose sinusoids because, for any fixed offset k, $PE(pos + k)$ is a linear function of $PE(pos)$, which they expected to make attention by relative position easy to learn.

13.3.7 Why self-attention

The paper compares layer types by cost per layer, parallelism, and the longest path a signal travels between two positions (n = sequence length, d = width, k = kernel size, r = neighborhood size).

Layer type	Complexity per layer	Sequential operations	Maximum path length
Self-attention	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
Recurrent	$O(n \cdot d^2)$	$O(n)$	$O(n)$
Convolutional	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(\log_k(n))$
Restricted self-attention	$O(r \cdot n \cdot d)$	$O(1)$	$O(n/r)$

Self-attention connects every pair of positions in a constant number of steps. The cost is quadratic in n, and the authors already proposed restricting attention to a neighborhood of size r

for very long sequences.

13.3.8 Training and decoding

- **Data:** WMT 2014 English–German (about 4.5 million sentence pairs, a shared byte-pair vocabulary of about 37,000 tokens) and English–French (36 million sentences, a 32,000 word-piece vocabulary).
- **Optimization:** Adam with $\beta_1 = 0.9$, $\beta_2 = 0.98$ and $\varepsilon = 10^{-9}$, with a learning rate that rises linearly for 4000 warm-up steps and then decays as the inverse square root of the step number.
- **Regularization:** dropout on sub-layer outputs and on the sums of embeddings and positional encodings (rate 0.1 for the base model), and label smoothing of 0.1, which the authors note hurts perplexity but improves accuracy and BLEU.
- **Decoding:** beam search with beam size 4 and length penalty 0.6, with maximum output length set to the input length plus 50 and early termination when possible.

13.3.9 Results

The big model reached 28.4 BLEU on WMT 2014 English–German, more than 2 BLEU above the best previous results, ensembles included, after 3.5 days on eight P100 GPUs. On English–French it set a single-model state of the art: 41.0 BLEU in the first arXiv version, reported as 41.8 in the revised version. A 4-layer model also performed strongly on English constituency parsing, showing that the architecture generalized beyond translation.

13.4 The Transformer as Operators and Measures

This section rewrites the model of Section 13.3 in the language of Section 13.2. The whole network is a composition of operators on finite-dimensional Hilbert spaces, written with $\circ$. Only its last factor has a codomain that is not a vector space: the set of probability measures on the σ-algebra of token events, from which the next token is sampled.

13.4.1 The Transformer as a composition of operators

The 2017 Transformer is a chain of maps, each with a stated domain and codomain. Every map but the last sends one finite-dimensional Hilbert space into another; the last sends a Hilbert space into the set of probability measures on the vocabulary.

Notation. $\mathcal{V}$ is the vocabulary, with $|\mathcal{V}| \approx 37{,}000$ shared byte-pair tokens for English–German. $d = d_{\text{model}} = 512$. The source sentence is $x = (x_1, \dots, x_n) \in \mathcal{V}^n$; the target prefix is $y_{<t} = (y_1, \dots, y_{t-1})$. All states are matrices in $\mathbb{R}^{n \times d}$, a Hilbert space under the Frobenius inner product (§13.2.1).

Building blocks. The table lists every operator in the order of evaluation.

Operator	Map	Definition	Type
One-hot encoding ι	$\mathcal{V}^n \to \mathbb{R}^{n \times \lvert\mathcal{V}\rvert}$	row i is the basis vector e_{x_i}	injective, measurable
Embedding ε	$\mathbb{R}^{n \times \lvert\mathcal{V}\rvert} \to \mathbb{R}^{n \times d}$	$\varepsilon(A) = \sqrt{d} \cdot AW_E$, with $W_E \in \mathbb{R}^{\lvert\mathcal{V}\rvert \times d}$	linear
Positional shift τ	$\mathbb{R}^{n \times d} \to \mathbb{R}^{n \times d}$	$\tau(X) = X + P_n$, with P_n the sinusoidal table	affine (a translation)

Operator	Map	Definition	Type
Multi-head attention M	$\mathbb{R}^{n\times d}\times\mathbb{R}^{m\times d}\to\mathbb{R}^{n\times d}$	$M(X;Y)=\sum_h \mathrm{softmax}(XW_h^Q (YW_h^K)^\top /\sqrt{d_k}+\mathrm{Mask})\, YW_h^V W_h^O$	nonlinear
Feed-forward F	$\mathbb{R}^{n\times d}\to\mathbb{R}^{n\times d}$	$F(X)=\max(0, XW_1+b_1)W_2+b_2$, row by row	piecewise linear
Layer normalization LN	$\mathbb{R}^{n\times d}\to\mathbb{R}^{n\times d}$	each row centred, divided by its standard deviation, then scaled and shifted	nonlinear, smooth
Row selection π_t	$\mathbb{R}^{m\times d}\to\mathbb{R}^{d}$	$\pi_t(Y)=$ row t of Y	linear
Logit map λ	$\mathbb{R}^{d}\to\mathbb{R}^{\lvert\mathcal{V}\rvert}$	$\lambda(h)=W_E h$, the same W_E as the embedding	linear
Softmax σ	$\mathbb{R}^{\lvert\mathcal{V}\rvert}\to\Delta(\mathcal{V})$	$\sigma(z)_v=e^{z_v}/\sum_u e^{z_u}$	nonlinear, into the simplex

The sum over heads in M is the concatenation of Section 13.3 written with W^O split into blocks W_h^O. Setting $Y=X$ gives self-attention; a mask of $-\infty$ above the diagonal makes it causal.

Sublayers as compositions. Each sublayer G is wrapped in a residual connection and a normalization. As an operator this is a composition:

$$R_G \;=\; \mathrm{LN}\circ(\mathrm{Id}+G), \qquad R_G(X)=\mathrm{LN}\big(X+G(X)\big).$$

Encoder. Encoder layer ℓ applies self-attention $S_\ell(X)=M_\ell(X;X)$ and then the feed-forward map F_ℓ. The embedding is $E=\tau\circ\varepsilon\circ\iota$. The encoder is the six-fold composition

$$\mathrm{Enc} \;=\; L_6^{\mathrm{enc}}\circ L_5^{\mathrm{enc}}\circ\cdots\circ L_1^{\mathrm{enc}}\circ E, \qquad L_\ell^{\mathrm{enc}}=R_{F_\ell}\circ R_{S_\ell}.$$

It maps $\mathcal{V}^n$ to $\mathbb{R}^{n\times d}$. Its output $H=\mathrm{Enc}(x)$ is called the memory.

Decoder. Decoder layer ℓ has three sublayers: causal self-attention $\tilde{S}_\ell$, cross-attention $C_\ell[H]=M_\ell(Y;H)$ reading the memory, and the feed-forward map. The memory enters as a fixed parameter of each decoder layer:

$$L_\ell^{\mathrm{dec}}[H] \;=\; R_{F_\ell}\circ R_{C_\ell[H]}\circ R_{\tilde{S}_\ell}, \qquad \mathrm{Dec}[H] \;=\; L_6^{\mathrm{dec}}[H]\circ\cdots\circ L_1^{\mathrm{dec}}[H]\circ E.$$

The whole model. The decoder reads the prefix shifted right by one position, $(\langle s\rangle, y_1,\dots,y_{t-1})$, where $\langle s\rangle$ is a start symbol. The next-token distribution is

$$p_\theta(\,\cdot\mid x, y_{<t}) \;=\; \big(\sigma\circ\lambda\circ\pi_t\circ\mathrm{Dec}[\mathrm{Enc}(x)]\big)\big(\langle s\rangle, y_1,\dots,y_{t-1}\big).$$

θ collects every weight matrix, bias and normalization parameter. Written out, one prediction is a composition of $3+6\cdot 2\cdot 2+3+6\cdot 3\cdot 2+3=69$ elementary maps (two embeddings of three maps each, every encoder and decoder sublayer counted as a residual map plus a normalization, then readout, logits and softmax).

Table 13.5: The Transformer as a chain of operators, from source tokens to a sampled token.

Stage	Feeds into
Source $x \in \mathcal{V}^n$	Embedding E
Embedding E	Encoder: six layers
Encoder: six layers	Memory H
Prefix $y_{<t} \in \mathcal{V}^{t-1}$	Embedding E
Embedding E	Decoder: six layers
Memory H	Decoder: six layers
Decoder: six layers	Row t: π_t
Row t: π_t	Logits: λ
Logits: λ	Softmax: σ
Softmax: σ	Measure on $\mathcal{V}$
Measure on $\mathcal{V}$	Sampled token y_t

Every factor before σ has a Hilbert space as its codomain. σ changes the type of object: its values are points of the simplex $\Delta(\mathcal{V})$, which §13.4.5 identifies with the probability measures on $(\mathcal{V}, 2^{\mathcal{V}})$. The last arrow is not a deterministic function of the measure: it requires an independent source of randomness (§13.4.5).

13.4.2 The operators one by one

The properties of the composite follow from those of its factors: linear maps contribute their operator norms, the nonlinear maps their local behavior, and the chain rule assembles them.

Linear and affine factors. ι, ε, π_t and λ are linear, and τ is a translation. Their Lipschitz constants are operator norms (§13.2.2): for instance ε is $\sqrt{d} \cdot \sigma_{\max}(W_E)$-Lipschitz, where $\sigma_{\max}$ is the largest singular value (§13.2.3). The input x itself is discrete; continuity questions start after ι, which embeds $\mathcal{V}^n$ into $\mathbb{R}^{n \times |\mathcal{V}|}$.

Layer normalization is a projection followed by a radial map. For a row $x \in \mathbb{R}^d$,

$$\mathrm{LN}(x) = \gamma \odot \frac{Px}{\sqrt{\|Px\|^2/d + \epsilon}} + \beta, \qquad P = I - \tfrac{1}{d}\mathbf{1}\mathbf{1}^\top.$$

P is the orthogonal projection onto the hyperplane orthogonal to the all-ones vector (the projection theorem of §13.2.1). The radial factor sends Px into the ball of radius $\sqrt{d}$ in that hyperplane. LN therefore maps all of $\mathbb{R}^d$ into a compact set, which is what makes the finite-precision arguments of §13.2.9 and §13.4.5 possible. LN is smooth because $\epsilon > 0$. On the hyperplane its Jacobian has norm at most $\max|\gamma|/\sqrt{\epsilon}$, so it is globally Lipschitz but with a very large constant for small ϵ; for rows with variance near 1 its local constant is about $\max|\gamma|$.

Feed-forward layers. $F = (\cdot W_2 + b_2) \circ \mathrm{ReLU} \circ (\cdot W_1 + b_1)$ is piecewise linear. ReLU is 1-Lipschitz, so F is $\|W_1\|_2\|W_2\|_2$-Lipschitz. In the language of §13.2.5, F belongs to $W^{1,\infty}_{\mathrm{loc}}$: its weak derivative exists and is bounded, and its curvature is concentrated on the hyperplanes where a ReLU unit changes sign.

Attention is real-analytic in its input, hence locally Lipschitz, but not globally Lipschitz (§13.4.3).

Residual maps. If G is L-Lipschitz, $\mathrm{Id} + G$ is $(1 + L)$-Lipschitz. Lipschitz constants multiply under composition, so the whole chain of §13.4.1 is Lipschitz on bounded sets with constant at most

the product of the 69 factors' constants. That bound is valid but far from sharp; the informative object is the Jacobian at a given input.

The chain rule for the whole model. Write the chain as $\Phi = \Phi_N \circ \cdots \circ \Phi_1$ with states $X_k = \Phi_k(X_{k-1})$. Wherever every factor is differentiable,

$$D\Phi(X_0) = D\Phi_N(X_{N-1})\, D\Phi_{N-1}(X_{N-2}) \cdots D\Phi_1(X_0), \qquad \nabla_{X_0}\ell = D\Phi(X_0)^\top \nabla\ell.$$

Backpropagation evaluates the second formula starting from the output: it applies $D\Phi_N^\top$ to $\nabla\ell$, then $D\Phi_{N-1}^\top$, and so on down to the input, multiplying transposed Jacobians onto a vector without ever forming a full Jacobian matrix. Because ReLU is not differentiable at zero, some care is needed. Rademacher's theorem (§13.2.2) makes the whole network differentiable almost everywhere, but it does not by itself guarantee that the product of layer Jacobians computed by backpropagation equals that derivative: a composition can sit on a kink of an inner unit on a set of positive measure (a dead unit, or the convention $\mathrm{ReLU}'(0) = 0$). For piecewise-analytic networks such as this one, backpropagation returns the true gradient almost everywhere and an element of a conservative field everywhere (Kakade and Lee, 2018; Bolte and Pauwels, 2021).

Post-norm changes the Jacobian of each block. For $R_G = \mathrm{LN} \circ (\mathrm{Id} + G)$,

$$DR_G(X) = D\mathrm{LN}\big(X + G(X)\big)\,\big(I + DG(X)\big).$$

The identity term is multiplied by the Jacobian of the normalization. A gradient reaching the encoder input passes through the normalization Jacobian of every sublayer on its path, and there is no identity path through the stack. Xiong et al. (2020) relate this to the 2017 model's dependence on learning-rate warm-up; the pre-norm form $X + G(\mathrm{LN}(X))$, whose Jacobian is $I + DG \cdot D\mathrm{LN}$, restores a direct identity path.

Summary of regularity. The composite map from the one-hot input to the logits is piecewise real-analytic: analytic on each region where the ReLU activation pattern is fixed, locally Lipschitz everywhere, differentiable almost everywhere, and in $W^{1,\infty}_{\mathrm{loc}}$. The softmax that follows is analytic, so every next-token probability is a locally Lipschitz function of the embedded input and of θ.

13.4.3 Attention as a data-dependent Markov kernel

Inside the network, probability measures already appear before the last layer: every attention head builds one measure over positions for each query, and outputs an expectation under it.

One head. For queries from $X \in \mathbb{R}^{n\times d}$ and keys and values from $Y \in \mathbb{R}^{m\times d}$, the head computes scores, a row-stochastic matrix, and values:

$$A = \frac{XW^Q\,(YW^K)^\top}{\sqrt{d_k}} + \mathrm{Mask}, \qquad P = \mathrm{softmax}_{\mathrm{row}}(A), \qquad \mathrm{head}(X;Y) = P\,YW^V.$$

Row i of P is a probability measure on the finite set of key positions $\{1,\dots,m\}$ with its power-set σ-algebra: a Markov kernel from query positions to key positions (§13.2.6). Row i of the output is an expectation:

$$\mathrm{head}(X;Y)_i = \sum_{j=1}^{m} P_{ij}\,(YW^V)_j = \mathbb{E}_{j\sim P_i.}\big[(YW^V)_j\big].$$

The kernel is recomputed from the input at every forward pass. This dependence is the source of attention's nonlinearity: for fixed P the map $Y \mapsto PYW^V$ would be linear.

Three kernels in the 2017 model.

Sublayer	Kernel	Shape	Support of row i
Encoder self-attention	source positions to source positions	$n \times n$, all entries positive	all of $\{1, \dots, n\}$
Decoder self-attention	target positions to target positions	$m \times m$, lower triangular	$\{1, \dots, i\}$
Cross-attention	target positions to source positions	$m \times n$, all entries positive	all of $\{1, \dots, n\}$

A causal mask sets scores above the diagonal to $-\infty$. By §13.2.7 this is conditioning: row i is the unmasked measure conditioned on the event $\{j \le i\}$.

Norms. Each output coordinate is a convex combination of value coordinates, so P is non-expansive in the maximum norm: $\|P\|_{\infty\to\infty} = 1$. In the Euclidean norm it is not: $\|P\|_2$ can reach $\sqrt{m}$ (§13.2.3).

Local but not global Lipschitz continuity. Each row of the softmax has Jacobian $\mathrm{diag}(p) - pp^\top$ (§13.2.7), and the scores are quadratic in the input. The head is therefore real-analytic and locally Lipschitz, but its sensitivity grows with the size of its input, and dot-product self-attention is not globally Lipschitz (Kim, Papamakarios and Mnih, 2021). In the post-norm 2017 model every attention sublayer after the first reads a layer-normalized state, which lies in a compact set (§13.4.2). On that set attention is Lipschitz. The factor $1/\sqrt{d_k}$ keeps the score variance near 1 when query and key components have unit variance, which keeps the softmax away from saturation.

Permutation equivariance of the encoder. Let Π be an $n \times n$ permutation matrix. With no mask, $\mathrm{softmax}_{\mathrm{row}}(\Pi A \Pi^\top) = \Pi\, \mathrm{softmax}_{\mathrm{row}}(A)\, \Pi^\top$, and therefore

$$S(\Pi X) = \Pi\, S(X), \qquad F(\Pi X) = \Pi F(X), \qquad \mathrm{LN}(\Pi X) = \Pi\, \mathrm{LN}(X).$$

Every encoder layer is permutation-equivariant. Only the positional translation τ breaks the symmetry, since P_n is not permutation-invariant: all information about word order in the encoder comes from τ. The decoder's causal mask is not permutation-invariant, so the decoder is not equivariant, and the mask itself carries order information.

Cross-attention depends on the memory only through its empirical measure. Permuting the rows of the memory H permutes keys and values together, so $C[\Pi H] = C[H]$. The output depends on H only through the empirical measure $\mu_H = (1/n)\sum_j \delta_{h_j}$ of its rows:

$$C[H](y) = \frac{\int e^{\langle yW^Q,\, hW^K\rangle/\sqrt{d_k}}\, hW^V \, d\mu_H(h)}{\int e^{\langle yW^Q,\, hW^K\rangle/\sqrt{d_k}}\, d\mu_H(h)}.$$

This is the Nadaraya–Watson form of the integral operator of §13.2.4 with the exponential kernel. The positive definiteness of §13.2.4, and with it Mercer's theorem, holds in the projected variables $q = yW^Q$ and $k = hW^K$. As a function of (y, h) itself the kernel is $e^{yBh^\top/\sqrt{d_k}}$ with $B = W^Q(W^K)^\top$, which is neither symmetric nor positive definite. Order in the source survives only because τ has written positions into the rows of H.

Stacking kernels. Composing attention layers multiplies row-stochastic matrices, as in a Markov chain. The Dobrushin coefficient $\delta(P) = \frac{1}{2}\max_{i,k}\sum_j |P_{ij} - P_{kj}|$ is submultiplicative, $\delta(PQ) \le \delta(P)\,\delta(Q)$, and it measures how far apart the rows of P are. Every entry of an unmasked softmax kernel is positive, so $\delta(P) < 1$: each unmasked layer strictly contracts the differences between rows. Pure attention stacks therefore drive all rows toward a common value and lose rank with depth (Dong, Cordonnier and Loukas, 2021). The residual maps $\mathrm{Id} + G$ and the feed-forward layers of §13.4.1 are what prevent this collapse.

13.4.4 Spectral structure: heads, weight tying and positional rotations

The nonlinear operators of §13.4.1 are built from linear maps, and those linear maps have the spectral structure of §13.2.3.

Each head compares tokens through a low-rank bilinear form. Before the softmax, head h scores a query row x against a key row y by

$$b_h(x,y) = \frac{x^\top B_h\, y}{\sqrt{d_k}}, \qquad B_h = W_h^Q (W_h^K)^\top \in \mathbb{R}^{512\times 512}, \qquad \operatorname{rank} B_h \le d_k = 64.$$

With the SVD $B_h = \sum_{r\le 64} s_r u_r v_r^\top$, the head compares the components of the query along u_r with those of the key along v_r, weighted by s_r. It depends only on the projections of the query and the key onto 64-dimensional subspaces of the 512-dimensional representation.

Asymmetry. Split B_h into symmetric and skew-symmetric parts, $B_h = \frac{1}{2}(B_h + B_h^\top) + \frac{1}{2}(B_h - B_h^\top)$. The skew part is why $b_h(x,y) \neq b_h(y,x)$ in general: token i can assign large weight to token j while j assigns small weight to i. The skew part is not the only source: the softmax normalizes each row separately, so the attention weights are asymmetric in general even when B_h is symmetric.

The output of each head lies in a low-rank subspace. The value and output maps combine into $C_h = W_h^V W_h^O \in \mathbb{R}^{512\times 512}$, also of rank at most 64. Multi-head attention is

$$M(X;Y) = \sum_{h=1}^{8} P_h(X;Y)\, Y\, C_h,$$

a sum of eight terms, each confined to the at most 64-dimensional row space of its C_h. This is the precise form of the paper's motivation that heads attend to information from different representation subspaces. By Eckart–Young (§13.2.3), the singular values of B_h and C_h measure how far each head can be compressed.

Scores are low rank; the kernel need not be. The score matrix of a head has rank at most 64, but the exponential inside the softmax acts entrywise and can raise the rank. The kernel P_h may be full rank even though its scores are not.

Weight tying and Riesz representation. The logit map uses the embedding matrix: $\lambda(h)_v = \langle e_v, h\rangle$, where e_v is row v of W_E. By the Riesz representation theorem (§13.2.1), every linear functional on $\mathbb{R}^d$ is an inner product with a unique vector. Tying weights means that the vector representing token v at the input is also the representer of the functional that scores v at the output.

Positional encoding is a rotation. Group the 512 coordinates of P_n into 256 pairs, pair $j = 0,\dots,255$ oscillating at frequency $\omega_j = 10000^{-2j/512}$. Write pair j as a complex number, $z_j(\mathrm{pos}) = e^{i\omega_j \cdot \mathrm{pos}}$, with i the imaginary unit. A shift by k positions is then multiplication by $e^{i\omega_j k}$:

$$PE(pos+k) = R(k)\, PE(pos), \qquad R(k) = \operatorname{diag}\left(R_{-\omega_0 k}, \dots, R_{-\omega_{255} k}\right),$$

with R_θ the 2×2 rotation by θ. The sign is negative because each pair is stored in the order $(\sin, \cos)$. $R(k)$ is orthogonal, so all its singular values are 1: this is the linear shift property the authors cite, and the shift theorem of §13.2.8. A single pair determines position only modulo its period $2\pi/\omega_j$; the geometric progression of periods, from 2π to $2\pi \cdot 10000^{510/512} \approx 60{,}611$, resolves positions over a range set by the longest period (§13.2.8).

Where position enters the score. Because τ adds P_n to the content, a score expands into four terms:

$$(x_i + p_i)^\top B\,(x_j + p_j) = x_i^\top B x_j + x_i^\top B p_j + p_i^\top B x_j + p_i^\top B p_j.$$

The last term depends on positions only, but in general on both i and j, not only on $i-j$: $p_i^\top BR(j-i)p_i$ still depends on p_i unless B commutes with the rotations. Relative position is available to the model but not built into the score. Later rotary encodings (Su et al., 2021) multiply queries and keys by rotations instead of adding, which makes every score depend on position only through $i-j$.

13.4.5 The last layer: a probability measure on the σ-algebra of token events

The last three operators of §13.4.1, π_t, λ and σ, turn a vector of the Hilbert space $\mathbb{R}^d$ into a probability measure on a finite measurable space. Sampling then turns that measure into a token. This section makes each step precise.

Step 1: logits. The decoder's last state at position t is $h_t = \pi_t(\mathrm{Dec}[H](\langle \mathrm{s}\rangle, y_1, \dots, y_{t-1})) \in \mathbb{R}^{512}$. The logit map gives one real number per token:

$$z = \lambda(h_t) = W_E\, h_t \in \mathbb{R}^{|\mathcal{V}|}, \qquad z_v = \langle e_v, h_t\rangle.$$

Each logit is a linear functional of h_t with Riesz representer e_v (§13.4.4). Adding the same constant to every logit changes nothing below, so logits are really coordinates in the quotient space $\mathbb{R}^{|\mathcal{V}|}/\mathbb{R}\mathbf{1}$ (§13.2.7).

Step 2: the measurable space of token events. The next token takes values in the sample space $\mathcal{V}$. Because $\mathcal{V}$ is finite, the natural σ-algebra is its power set:

$$(\mathcal{V},\ 2^{\mathcal{V}}), \qquad |2^{\mathcal{V}}| = 2^{|\mathcal{V}|} \approx 2^{37\,000}.$$

Every subset of the vocabulary is an event, closed under complement and union as §13.2.6 requires. The atoms are the singletons $\{v\}$, and every event is a finite union of atoms. Events have direct linguistic readings:

- $\{v : v \text{ is a punctuation token}\}$, "the next token is punctuation";
- $\{v : v \text{ begins a new word}\}$, read off the byte-pair markers;
- $\{y_t\}$, "the model produces the reference token";
- the top-k set of the current logits, an event chosen by the logits themselves.

No measurability issues arise on this side: on a finite set every function is measurable. The continuous states of the network form the domain of the kernel, $\mathbb{R}^d$ with its Borel σ-algebra.

Step 3: the probability measure. The softmax assigns a mass to each atom, and additivity extends it to every event:

$$P_t(S) \;=\; \sum_{v\in S} \sigma(z)_v \;=\; \sum_{v\in S} \frac{e^{z_v}}{\sum_{u\in\mathcal{V}} e^{z_u}}, \qquad S \in 2^{\mathcal{V}}.$$

The axioms hold directly: $P_t(S) \ge 0$, $P_t(\mathcal{V}) = 1$, and $P_t(S \cup S') = P_t(S) + P_t(S')$ for disjoint S and S'. Since the σ-algebra is finite, countable additivity reduces to finite additivity. Every atom has positive mass, so the only null event is $\emptyset$: P_t has the same null sets as the counting measure.

The simplex is the space of measures. A probability measure on $(\mathcal{V}, 2^{\mathcal{V}})$ is determined by its values on atoms. The map $p \mapsto \sum_v p_v \delta_v$ identifies the simplex $\Delta(\mathcal{V}) = \{p \in \mathbb{R}^{|\mathcal{V}|} : p \ge 0, \sum_v p_v = 1\}$ with the set of all probability measures on $(\mathcal{V}, 2^{\mathcal{V}})$. The softmax is a bijection from $\mathbb{R}^{|\mathcal{V}|}/\mathbb{R}\mathbf{1}$ onto the open simplex, the measures that charge every atom; its inverse is $p \mapsto \log p$ up to a constant. The logits are thus the natural parameters of the categorical exponential family, and σ is the operator of §13.4.1 that changes the type of object: its codomain is a convex set of measures, not a vector space.

Step 4: the whole model is a Markov kernel. Write Φ_θ for the deterministic part, the map from the context $(x, y_{<t})$ to h_t, and $\kappa_\theta(h, S) = \sum_{v \in S} \sigma(W_E h)_v$ for the last layer. κ_θ is a Markov kernel from $(\mathbb{R}^d, \text{Borel})$ to $(\mathcal{V}, 2^{\mathcal{V}})$: for each h it is a probability measure, and for each event S the map $h \mapsto \kappa_\theta(h, S)$ is continuous, hence Borel measurable (§13.2.6). The model's conditional law is the kernel composed with the operator chain:

$$P_\theta(S \mid x, y_{<t}) \;=\; \kappa_\theta(\Phi_\theta(x, y_{<t}),\, S), \qquad \Phi_\theta = \pi_t \circ \mathrm{Dec}[\mathrm{Enc}(x)].$$

Every factor of Φ_θ is a deterministic map between Hilbert spaces; randomness enters only through κ_θ.

Operations on the measure. Decoding procedures act on P_t through the structure of §13.2.7:

Operation	On the logits	On the measure
Temperature τ	$z \mapsto z/\tau$	moves along the Gibbs family; the maximizer of $\langle p, z\rangle + \tau H(p)$
$\tau \to 0$	—	$P_t \to \delta$ at the most likely token, when it is unique
$\tau \to \infty$	—	$P_t \to$ the uniform measure on $\mathcal{V}$
Top-k or nucleus restriction to S	$z_v \mapsto -\infty$ for $v \notin S$	conditioning: $P_t(\cdot \mid S) = P_t(\cdot \cap S)/P_t(S)$

Step 5: sampling a token. A measure is not a token. Sampling means constructing a random variable whose law is P_t. Take a probability space carrying one uniform variable, ($[0, 1]$, Borel sets, Lebesgue measure), order the vocabulary as $v_1, \ldots, v_K$, and let $F_k = \sum_{j \le k} P_t(\{v_j\})$ with $F_0 = 0$. Define

$$Y_t(u) = v_k \quad \text{when} \quad F_{k-1} < u \le F_k.$$

Y_t is measurable, because the preimage of each atom $\{v_k\}$ is the interval $(F_{k-1}, F_k]$. Its law, the pushforward of Lebesgue measure, is exactly P_t:

$$\mathrm{Leb}(Y_t^{-1}(\{v_k\})) = F_k - F_{k-1} = P_t(\{v_k\}).$$

The Gumbel-max construction gives the same law without normalizing: with G_v independent standard Gumbel variables, $\arg\max_v(z_v + G_v)$ has law $\sigma(z)$ (§13.2.7). In both constructions the randomness is external and independent of θ: the model supplies the measure, and a separate source of noise supplies the draw.

What training asks of the measure. During training the reference token y_t defines a target measure. With label smoothing $\varepsilon = 0.1$, it is

$$q_t = (1 - \varepsilon)\, \delta_{y_t} + \varepsilon\, u_{\mathcal{V}}, \qquad \mathcal{L}_t = H(q_t, P_t) = H(q_t) + \mathrm{KL}(q_t \,\|\, P_t),$$

with $u_{\mathcal{V}}$ the uniform measure. (Implementations differ in detail; some spread ε over the $|\mathcal{V}| - 1$ non-reference tokens only. The argument below is unchanged.) By Gibbs' inequality (§13.2.10) the loss is minimized at $P_t = q_t$, a measure that charges every atom; smoothing thus keeps the target inside the open simplex, which is the range of the softmax. How training selects θ is outside the scope of this chapter.

In floating point. The exact P_t charges every atom, but computed probabilities can underflow to zero, so the computed measure can have null events the ideal one lacks. Implementations compute $\log \sigma(z) = z - \max z - \log \sum_u e^{z_u - \max z}$, using the shift invariance of Step 1 to avoid overflow.

13.4.6 Sequences: from one token to a measure on translations

Section IV.5 produced one measure per position. Chained along the target sentence, these measures define a single probability measure on the space of whole translations, conditional on the source.

The sample space of translations. Fix a source sentence x. Outputs are sequences $\omega = (y_1, y_2, \ldots)$ in $\mathcal{V}^{\mathbb{N}}$; a finite translation is a sequence that reaches the end symbol $\langle/\mathrm{s}\rangle$ and stays there. The σ-algebra is generated by the cylinder sets of §13.2.6,

$$C(y_1, \ldots, y_t) = \{\omega \in \mathcal{V}^{\mathbb{N}} : \omega_1 = y_1, \ldots, \omega_t = y_t\}, \qquad \mathcal{F}_t = \sigma(\omega_1, \ldots, \omega_t),$$

and the filtration $\mathcal{F}_0 \subset \mathcal{F}_1 \subset \mathcal{F}_2 \subset \cdots$ records what is known after t tokens. The event "the translation begins with Das Haus" is a cylinder of length 2 and belongs to $\mathcal{F}_2$.

Predictability: the causal mask as a measurability statement. The decoder input at position s is y_{s-1}, shifted right by one. By induction over the layers of §13.4.1, row t of every decoder layer depends only on the inputs at positions $1, \ldots, t$ and on the memory H:

- causal self-attention: row t of its kernel is supported on $\{1, \ldots, t\}$ (§13.4.3);
- cross-attention: it reads $H = \mathrm{Enc}(x)$, which depends on x alone;
- feed-forward layers, normalization and residual maps act row by row.

So h_t is a function of $(x, y_1, \ldots, y_{t-1})$. By the Doob–Dynkin lemma (§13.2.6), the next-token kernel is $\mathcal{F}_{t-1}$-measurable for fixed x:

$$y \mapsto p_\theta(\,\cdot \mid x, y_{<t}) \text{ is } \mathcal{F}_{t-1}\text{-measurable for each fixed } x.$$

The sequence of kernels is predictable: each distribution is fixed before its token is drawn. The encoder's output, by contrast, depends on the whole source at once. The decoder's causal mask and the encoder's unmasked self-attention are thus two measurability conditions.

One measure on all translations. By the Ionescu-Tulcea theorem (§13.2.6), the predictable kernels determine a unique probability measure $P_\theta(\cdot \mid x)$ on $(\mathcal{V}^{\mathbb{N}}, \mathcal{F}_\infty)$. On cylinders it is given by the chain rule:

$$P_\theta(C(y_1, \ldots, y_t) \mid x) = \prod_{s=1}^{t} p_\theta(y_s \mid x, y_{<s}).$$

The cylinders form a π-system generating $\mathcal{F}_\infty$, so by the π-λ theorem these values determine $P_\theta(\cdot \mid x)$ uniquely. Letting x vary, $x \mapsto P_\theta(\cdot \mid x)$ is a Markov kernel from source sentences to target sequences: this kernel is the translation model.

Generation realizes the measure path by path. Draw independent uniforms $U_1, U_2, \ldots$ and set $Y_t = G(p_\theta(\cdot \mid x, Y_{<t}), U_t)$, with G the inverse-distribution construction of §13.4.5. Each Y_t is $\sigma(U_1, \ldots, U_t)$-measurable, and the law of $(Y_1, Y_2, \ldots)$ is exactly $P_\theta(\cdot \mid x)$. Equivalently, generation is a Markov chain on the countable set of prefixes $\mathcal{V}^*$, whose transitions append one token until $\langle/\mathrm{s}\rangle$ is emitted. States ending in $\langle/\mathrm{s}\rangle$ are absorbing: once the end symbol appears the chain stays put, which amounts to a cemetery state that emits nothing further.

Termination. The first time the end symbol appears, $T = \inf\{t : Y_t = \langle/\mathrm{s}\rangle\}$, is a stopping time: $\{T = t\} \in \mathcal{F}_t$. Generation ends almost surely exactly when $P_\theta(T < \infty \mid x) = 1$, the tightness

condition; in Markov-chain terms, the absorbing state is reached with probability one. The 2017 decoder also capped output length at the input length plus 50, which makes termination certain in practice.

The target of the kernel. Let P^* be the true joint law of source and target sentences. The ideal next-token law is a conditional expectation, and by §13.2.6 an orthogonal projection:

$$P^*(Y_t = v \mid x, Y_{<t}) = \mathbb{E}^*\big[\mathbf{1}_{\{Y_t=v\}} \,\big|\, \sigma(x) \vee \mathcal{F}_{t-1}\big],$$

the projection of the indicator of the event $\{Y_t = v\}$ onto the subspace of $L^2(P^*)$ of functions of the source and the prefix. By Doob–Dynkin it is a function of $(x, y_{<t})$, and the decoder is a parametrized family of exactly such functions. The divergence between the true and the model measures on whole sequences decomposes along the filtration as a sum of per-token divergences (§13.2.10), which is why a per-token loss trains a sequence model.

13.4.7 Decoding and the idealized model

The measure $P_\theta(\cdot \mid x)$ is the model; a decoding rule is a separate procedure applied to it. The 2017 paper decoded by search rather than by sampling.

Sampling versus search. Three rules act on the same measure:

Rule	Output	Relation to $P_\theta(\cdot \mid x)$
Ancestral sampling (§13.4.6)	a random translation	a draw whose law is exactly $P_\theta(\cdot \mid x)$
Greedy decoding	$\arg\max_v p_\theta(v \mid x, y_{<t})$ at each step	the $\tau \to 0$ limit of per-step sampling; not in general the most likely sequence
Beam search, beam 4, length penalty $\alpha = 0.6$	an approximate maximizer of $\log P_\theta(y \mid x)/\mathrm{lp}(y)$	a deterministic functional of the measure, not a draw from it

Beam search keeps the four best prefixes at each step and compares finished candidates after dividing their log-probability by a length penalty $\mathrm{lp}(y)$ that grows with length, so that shorter outputs are not automatically preferred. It is a heuristic for the mode of a reweighted sequence measure. The BLEU scores of Section 13.3 therefore measure a mode-seeking functional of P_θ, not typical samples from it.

The model that was computed. The analysis of Parts I and IV concerns the exact operators. Three features of the 2017 setup separate the computed model from that idealization:

- **Finite precision.** Every operator was evaluated in floating point on P100 GPUs, so the states lie on a finite grid and small probabilities can underflow (§13.2.9, §13.4.5).
- **Dropout.** During training, dropout (rate 0.1 for the base model, 0.3 for the big English–German model) multiplies sublayer outputs by random masks. The training-time network is therefore itself a random operator, a Markov kernel from inputs to states, and only the test-time network, with dropout off, is the deterministic composition of §13.4.1.
- **Finite length.** The decoder never runs beyond input length plus 50 tokens, so the measure actually realized is the image of $P_\theta(\cdot \mid x)$ under truncation at that length.

The chain as a whole. Each stage of the model lives on a different mathematical object, and each passage between stages is one of the operators or constructions of Section 13.2:

Stage	Object	Space	Structure used
Input	source x, prefix $y_{<t}$	$\mathcal{V}^n$, $\mathcal{V}^{t-1}$	finite sets, power-set σ-algebras
Embedding	$E = \tau \circ \varepsilon \circ \iota$	$\mathbb{R}^{n\times d}$	affine map into a Hilbert space
Encoder and decoder	12 layers composed with $\circ$	$\mathbb{R}^{n\times d}$, $\mathbb{R}^{m\times d}$	nonlinear operators; attention kernels inside
Readout	$\lambda \circ \pi_t$	$\mathbb{R}^{\lvert\mathcal{V}\rvert}/\mathbb{R}\mathbf{1}$	linear functionals, Riesz representers
Softmax	σ	$\Delta(\mathcal{V})$	probability measures on $(\mathcal{V}, 2^{\mathcal{V}})$
Sampling	Y_t	$\mathcal{V}$	a random variable with law P_t
Sequence	$P_\theta(\cdot \mid x)$	$(\mathcal{V}^{\mathbb{N}}, \mathcal{F}_\infty)$	filtration, predictable kernels, Ionescu-Tulcea

Read from top to bottom, this is the thesis of the chapter: the Transformer is a composition of operators on Hilbert spaces, followed by a single map into the space of probability measures on the σ-algebra of token events; sampling, applied step after step along a filtration, turns that measure into text.

13.5 Beyond the Structure of the Family

A transformer, as described in this chapter, is a map $\theta \mapsto P_\theta$ from parameters to measures on sequence space. The geometry of Hilbert spaces carries its representations; spectral theory governs its parameters and local behavior; measure-theoretic probability, organized by the σ-algebra of events and its filtration, turns its output into distributions over tokens and sequences. Everything here concerns the structure of that family. The natural next question, which this chapter leaves open, is which member to choose: pretraining as maximum likelihood, fine-tuning, reinforcement learning with a KL penalty, whose optimum is a Gibbs tilt of a reference measure, and on-policy distillation between sequence measures.

13.6 Sources

Primary sources:

- Vaswani et al., Attention Is All You Need, arXiv:1706.03762 (first version June 2017; results quoted from v1 unless noted).

Further references:

- Approximation theory: Cybenko (1989); Leshno, Lin, Pinkus and Schocken (1993); Barron (1993); Yarotsky (2017, 2018); DeVore, Howard and Micchelli; Suzuki (2019); DeVore, Hanin and Petrova, *Neural Network Approximation*, Acta Numerica (2021).
- Layer normalization placement: Xiong et al., *On Layer Normalization in the Transformer Architecture*, ICML (2020).

- Rank collapse in attention stacks: Dong, Cordonnier and Loukas, *Attention is not all you need: pure attention loses rank doubly exponentially with depth*, ICML (2021).
- Kernel methods: Steinwart and Christmann, *Support Vector Machines*, for Mercer's theorem and eigenvalue decay.
- Lipschitz continuity of attention: Kim, Papamakarios and Mnih, *The Lipschitz Constant of Self-Attention*, ICML (2021).
- Differentiating nonsmooth networks: Kakade and Lee, *Provably Correct Automatic Subdifferentiation for Qualified Programs*, NeurIPS (2018); Bolte and Pauwels, *Conservative Set Valued Fields, Automatic Differentiation, Stochastic Gradient Methods and Deep Learning*, Mathematical Programming (2021).
- Rotary position encodings: Su et al., *RoFormer: Enhanced Transformer with Rotary Position Embedding*, arXiv:2104.09864 (2021).

Conclusion

The four-layer architecture this book describes: intelligence, action, memory, knowledge, and security, represents the foundation of agentic systems.

The field is moving fast. Open-weight models are closing the gap with closed frontier models on the dimensions that matter most for agentic tasks, enabling a democratisation of the technology that will broaden access to production-grade agent capabilities across organisations and research communities.

The harder problem is reliability and governance: building systems that act autonomously on behalf of people and organisations with the same engineering rigour applied to traditional machine learning software. Evaluation, observability, and principled deployment pipelines are the infrastructure that makes that possible. They are also the least glamorous part of the work, and the part most often deferred.

The reader who has followed this book to its end has the foundation to build agents that hold up not only in proof of concept but in deployment.